

Instituto Tecnológico de Costa Rica

Profesor:

Gerardo Nereo Campos Araya

Estudiantes:

Manuel Calero Ríos 2022082503

Brandon Mora Díaz 2022164409

Kendall Rodríguez Camacho 2022049438

Fabían Vargas Cascante 2018160243

Curso:

Bases de Datos II

Fecha de entrega:

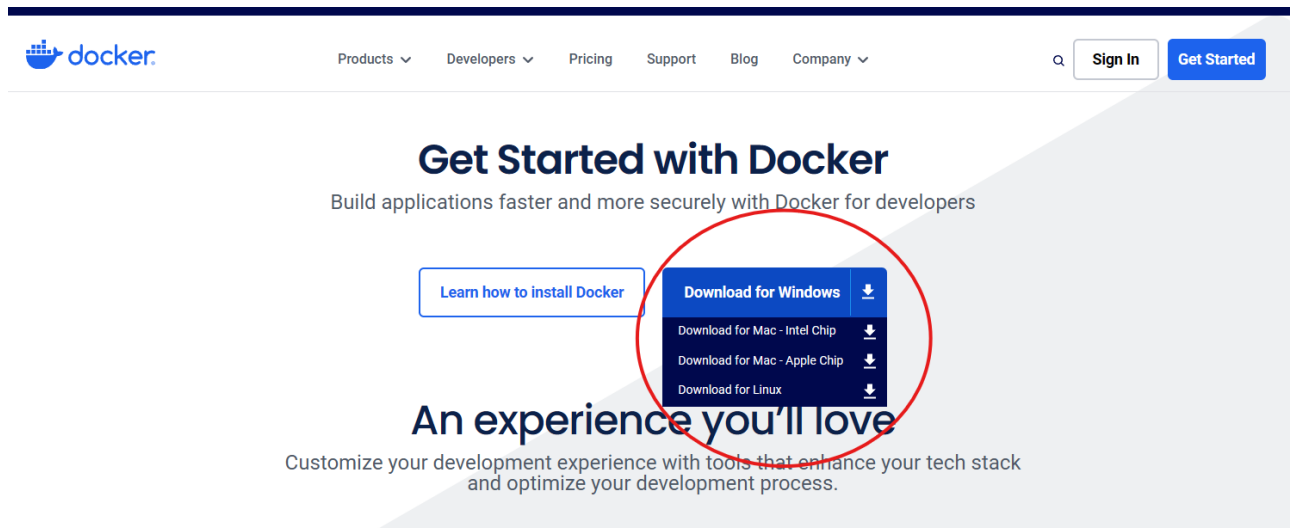
24 de mayo del 2024

Componentes implementados

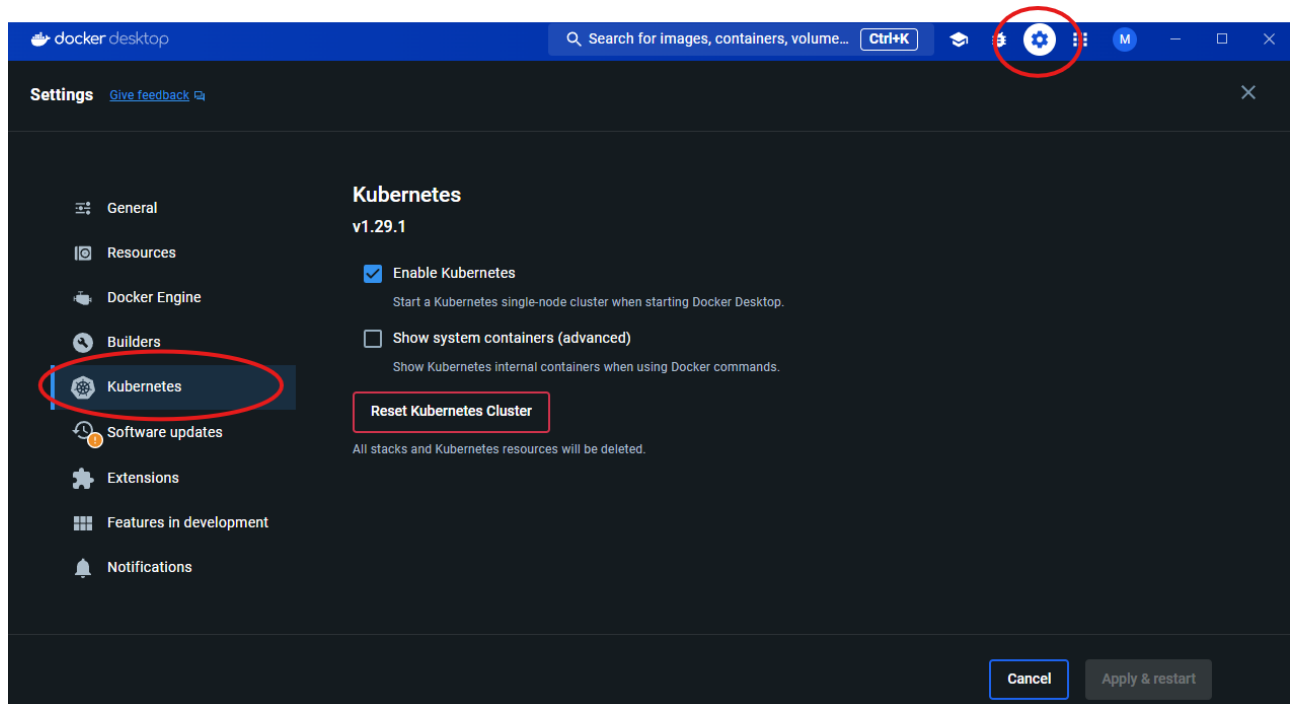
Para el desarrollo y ejecución del proyecto, hace falta tener instaladas ciertas aplicaciones esenciales, a continuación se nombran cuales son junto a sus métodos de instalación:

1. Docker:

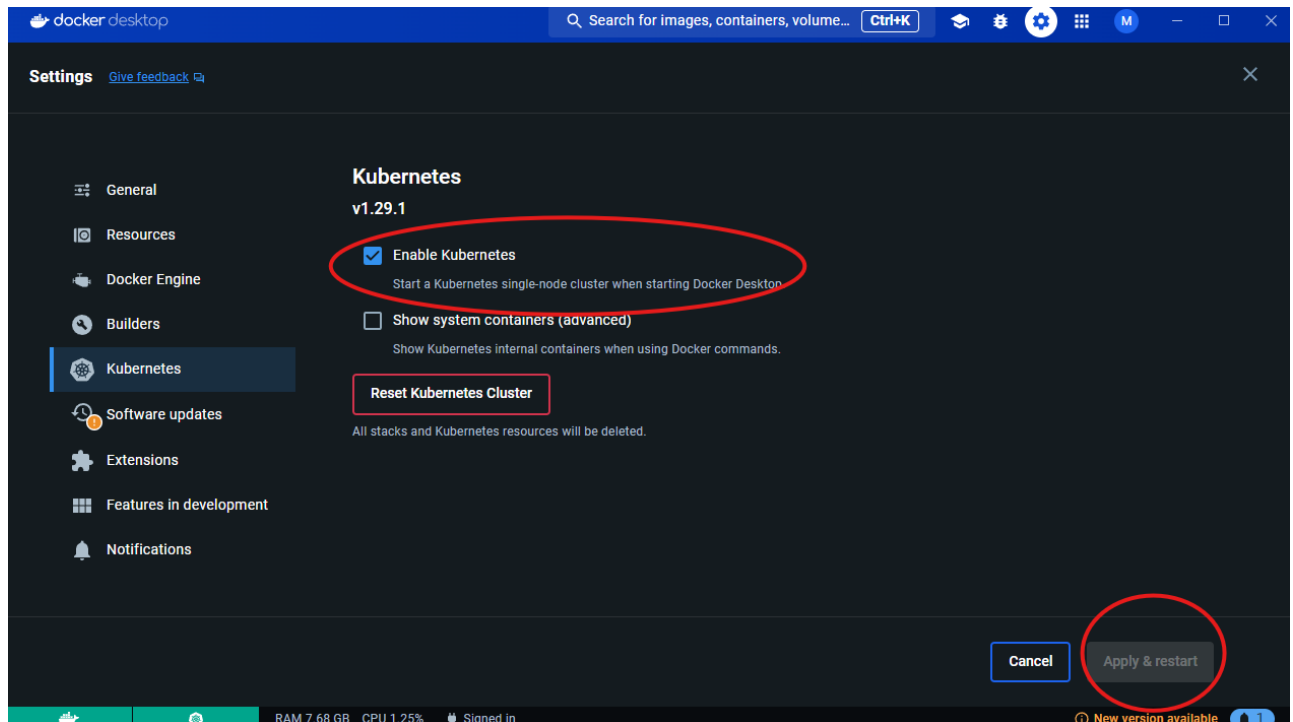
- Ingresar al siguiente link: <https://www.docker.com/get-started/>
- Una vez dentro, seleccionar el dispositivo en el que será utilizado y descargar el .exe. Además de ello deberá de seleccionar la opción de "Get Started" y crear una cuenta para poder hacer uso de Docker.



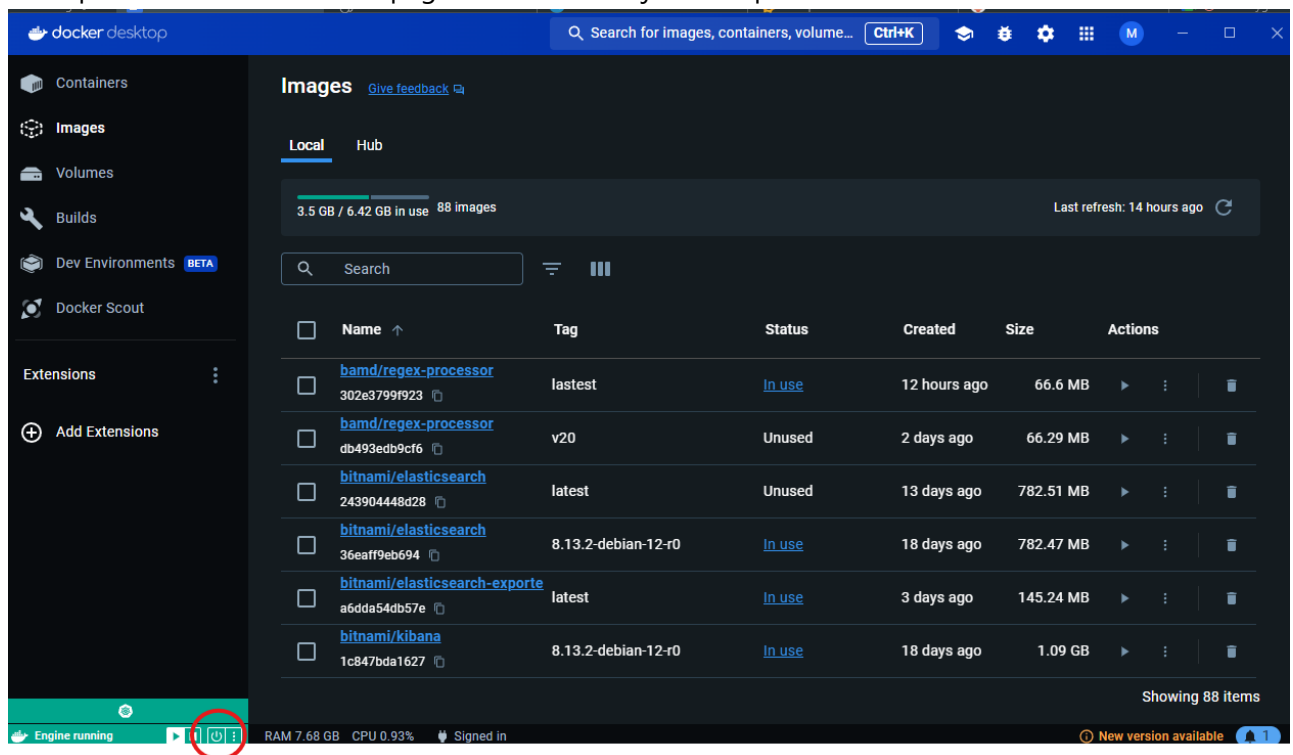
- Durante la instalación, debe darle continuar a todo, una vez termine la instalación, Docker iniciará y le desplegará una encuesta, además le pedirá que inicie sesión nuevamente.
- Una vez completados esos pasos, solo hace falta instalar Kubernetes, para ello seleccione la tuerca de arriba a la derecha. Una vez dentro seleccione la opción de la izquierda llamada Kubernetes



- Una vez seleccionado, se debe seleccionar la opción de "Enable Kubernetes" y presionar la opción de abajo a la derecha.



- Una vez seleccionado, toca esperar unos minutos a que finalice la instalación. Una vez finalizada se debe reiniciar el Docker o bien la computadora para que Kubernetes funcione. Para reiniciar Docker, debe presionar el símbolo de apagado ubicado abajo a la izquierda.



2. WinRAR:

- Ingresar al siguiente link: <https://www.winrar.es/descargas>
- Puede usar el descompresor de archivos de su preferencia, en este caso usaremos WinRAR para el ejemplo. Una vez ingresado al link, debe descargar el archivo que más se adapte a sus necesidades.



[Mi cuenta](#) | [¿ Quienes somos ?](#) | [Contactar](#)

[Inicio](#)
[Descargas](#)
[Comprar](#)
[Soporte](#)
[Noticias](#)

Descarga gratuita recomendada

Descargue WinRAR ahora
WinRAR 7.00 para Windows x64 Español (3,90Mb)

↓

Su sistema operativo: Windows 10 x64
Sus idiomas preferidos: Español, Inglés
Descarga recomendada: [winrar-x64-700es.exe](#)

Aviso de nuevas versiones

Puede activar el aviso de nuevas versiones de forma anónima en [este enlace](#). No hace falta su email.

Últimas descargas

 WinRAR 7.00 para Windows x64 en Español Martes, 19 De Marzo De 2024 A Las 09:50	 WinRAR 7.00 para Windows en Español Martes, 19 De Marzo De 2024 A Las 09:50
 WinRAR 7.00 para Windows x64 en Catalán Martes, 19 De Marzo De 2024 A Las 09:50	 WinRAR 7.00 para Windows en Catalán Martes, 19 De Marzo De 2024 A Las 09:50
 WinRAR 7.00 para Windows x64 en Euskera Martes, 19 De Marzo De 2024 A Las 09:50	 WinRAR 7.00 para Windows en Euskera Martes, 19 De Marzo De 2024 A Las 09:50
 WinRAR 7.00 para Windows x64 en Gallego Martes, 19 De Marzo De 2024 A Las 09:50	 WinRAR 7.00 para Windows en Gallego Martes, 19 De Marzo De 2024 A Las 09:50
 WinRAR 7.00 para Windows x64 en Inglés Miércoles, 28 De Febrero De 2024 A Las 10:48	 WinRAR 7.00 para Windows en Inglés Miércoles, 28 De Febrero De 2024 A Las 10:48

Descargas

Descarga recomendada

[WinRAR](#)

[Temas para WinRAR](#)

[RAR](#)

[RAR para movil](#)

[UnRAR](#)

[Fuentes y librerías](#)

- Una vez descargada solo debe de instalarla. Ya con eso tendrá el descompresor de datos listo para ser utilizado.

3. Helm:

- Ingresar al siguiente link: <https://helm.sh/docs/intro/install/>
- Una vez dentro, debe bajar y seleccionar "release"

From The Helm Project

The Helm project provides two ways to fetch and install Helm. These are the official methods to get Helm releases. In addition to that, the Helm community provides methods to install Helm through different package managers. Installation through those methods can be found below the official methods.

From the Binary Releases

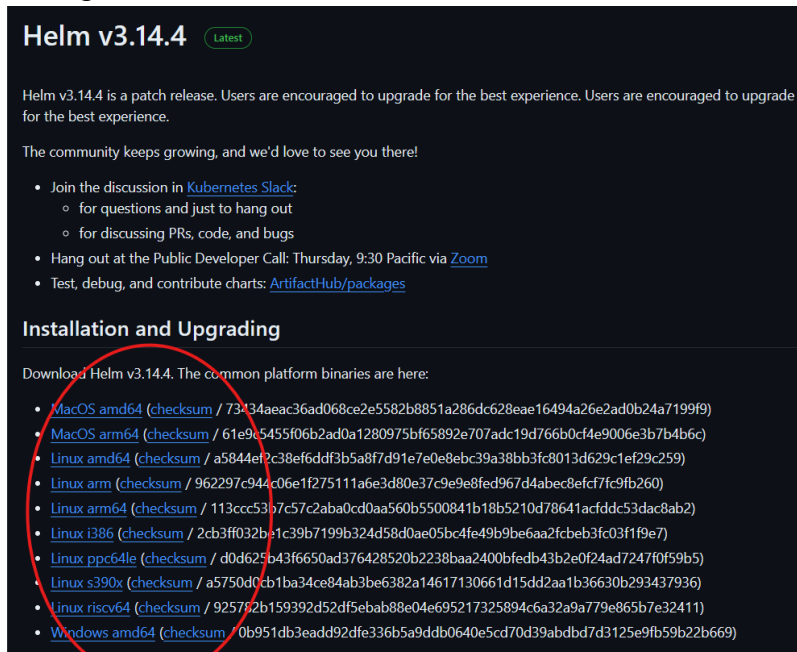
Every **release** of Helm provides binary releases for a variety of OSes. These binary versions can be manually downloaded and installed.

1. Download your [desired version](#)
2. Unpack it (`tar -zxvf helm-v3.0.0-linux-amd64.tar.gz`)
3. Find the `helm` binary in the unpacked directory, and move it to its desired destination (`mv linux-amd64/helm /usr/local/bin/helm`)

From there, you should be able to run the client and [add the stable repo](#) : `helm help` .

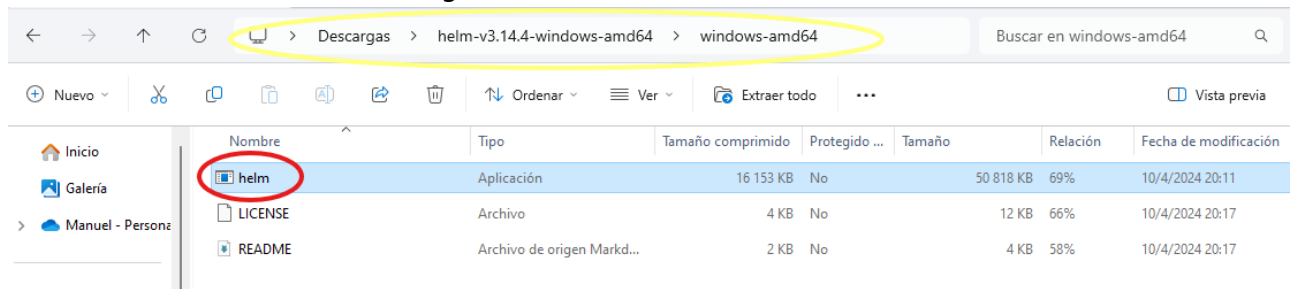
Note: Helm automated tests are performed for Linux AMD64 only during CircleCi builds and releases. Testing of other OSes are the responsibility of the community requesting Helm for the OS in question.

- Al entrar, se redirigirá a GitHub, ahí saldrán las versiones de Helm, debe seleccionar la adecuada para su



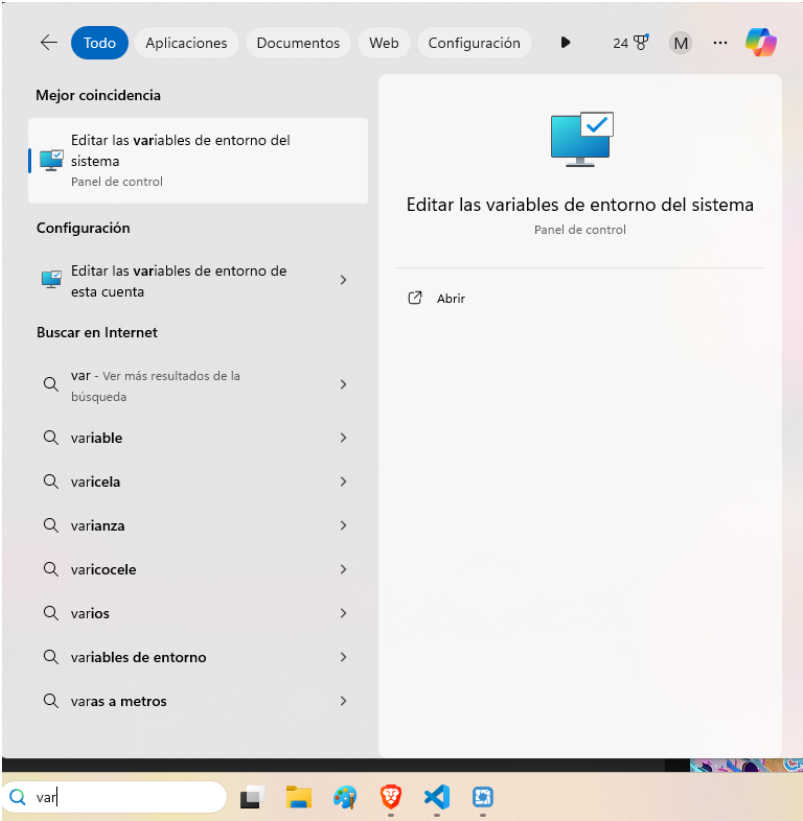
dispositivo.

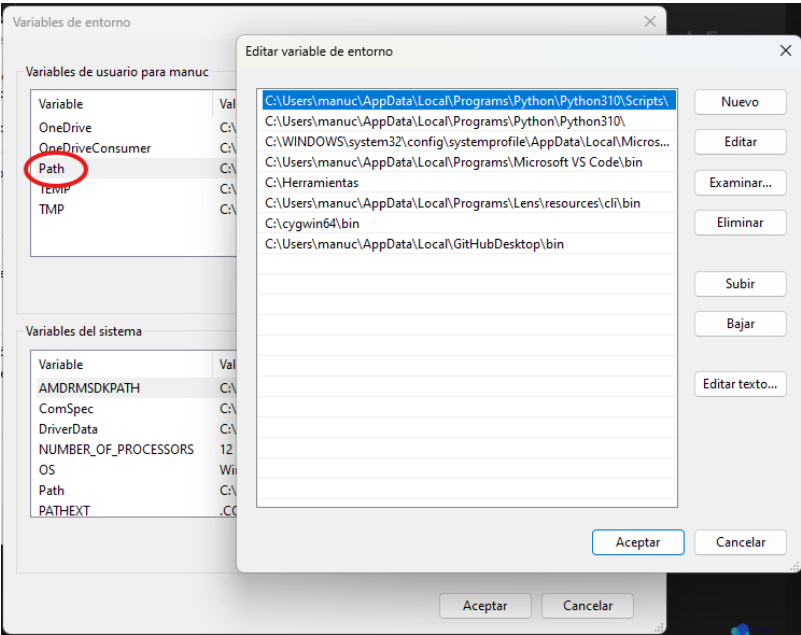
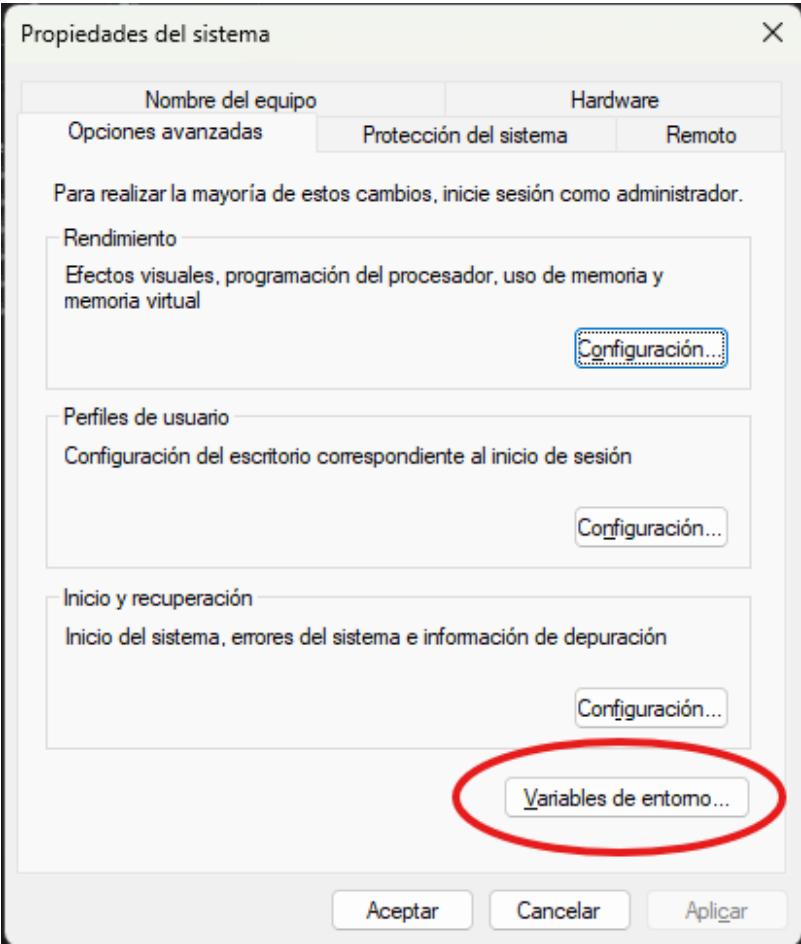
- Una vez descargado, debe descomprimir el archivo. Una vez hecho debe abrir la carpeta y buscar el archivo helm.exe. Una vez localizado, debe obtener la ruta o ubicación del sitio donde está guardado el helm.exe. La marca amarilla es una guía de dónde estará la dirección del helm.exe.



- Ahora, debe buscar el editor de variables del sistema de su dispositivo. Una vez dentro debe buscar el elemento llamado Path e incluir la dirección del sitio donde está almacenado el helm.exe, dicho proceso debe realizarse tanto en el Path de arriba, como el de abajo. Una vez realizado le da en aplicar

y aceptar todo. Al realizar esto, tendrá instalado Helm y listo para usar por comandos.





4. Lens:

- Ingresar al siguiente link: <https://k8slens.dev/>

- Una vez dentro, seleccionar la opción de descargar para su dispositivo

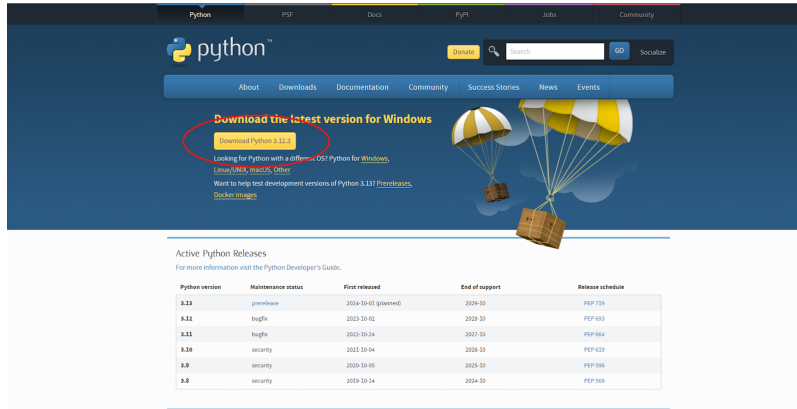


- Una vez dentro le va a pedir que inicie sesión, cuando lo hace ya tendría listo para utilizar Lens. No necesita configurar nada, pero si necesita ingresar a otros sitios, pero más adelante se le explicará.

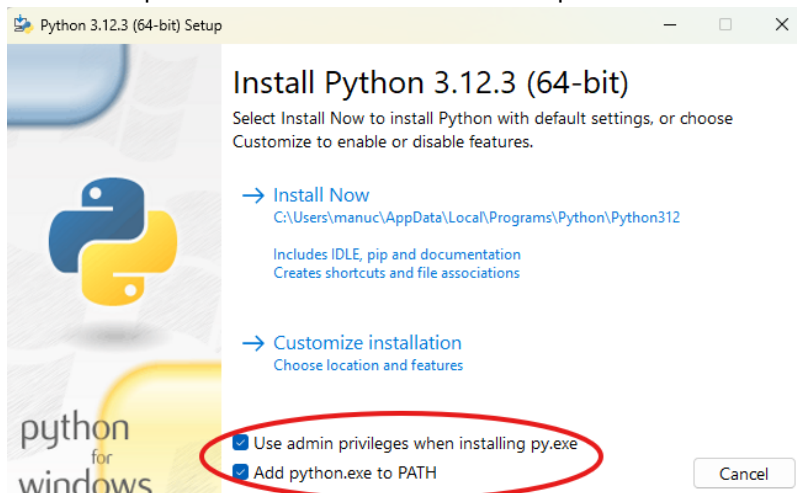
•

5. Python:

- Ingresar al siguiente link: <https://www.python.org/downloads/>
- Una vez dentro, le da a descargar la última versión.



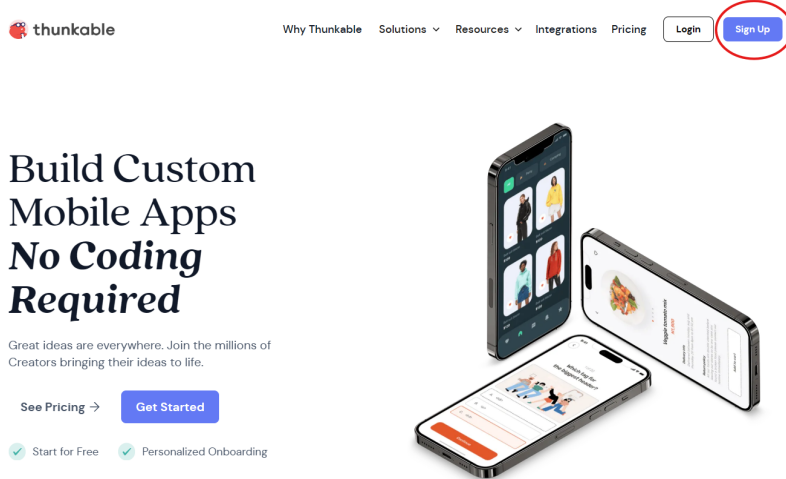
- Durante el proceso de instalación, solo es importante marcar las siguientes opciones.



- Una vez seleccionadas, continua con el proceso de instalación, dándole continuar hasta que termine la instalación.

6. Thunkable:

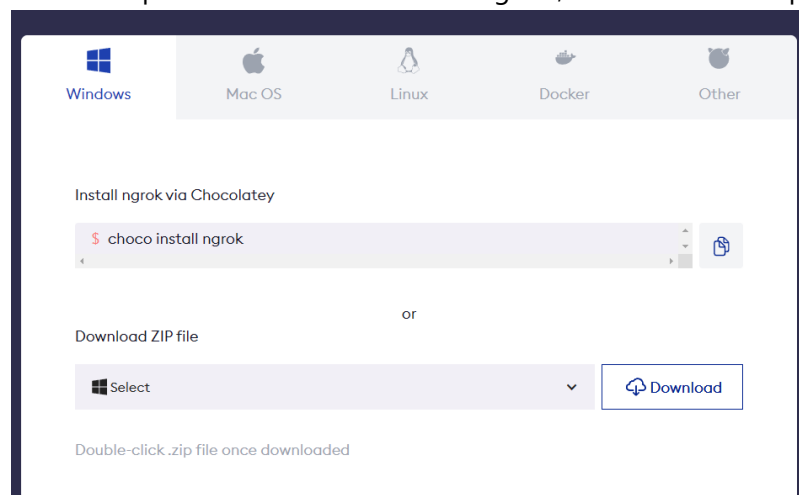
- Ingresar al siguiente link: <https://thunkable.com/>
- Una vez dentro, debe iniciar sesión en donde indica el círculo rojo.



- Con eso se inicia sesión en la página, ahora se ingresa al siguiente link para ingresar a la zona donde se realizan las pruebas vía interfaz gráfica:
<https://x.thunkable.com/copy/f3a0df8f2d2f7a1f7faf573ace432c57>

7. Ngrok:

- Ingrese al siguiente link: <https://ngrok.com/download>
- Una vez dentro, seleccione el dispositivo donde desea descargarlo, en este caso será por Windows



usando su instalador.

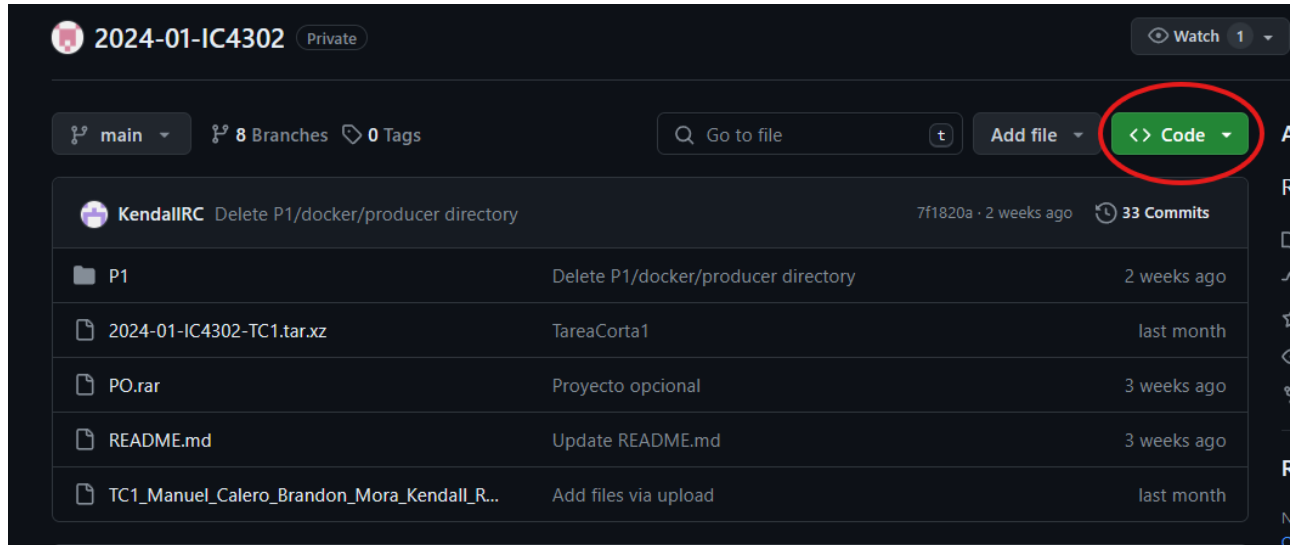
- Cuando se descargue, se tendrá un archivo .rar, el cual debe descomprimirse y buscar el archivo con nombre ngrok.exe. Una vez se ubique, debe realizar el mismo proceso que se hizo con helm, pero para evitar complicaciones puede colocar el ngrok.exe en la misma carpeta que helm.exe.

Instrucciones de ejecución

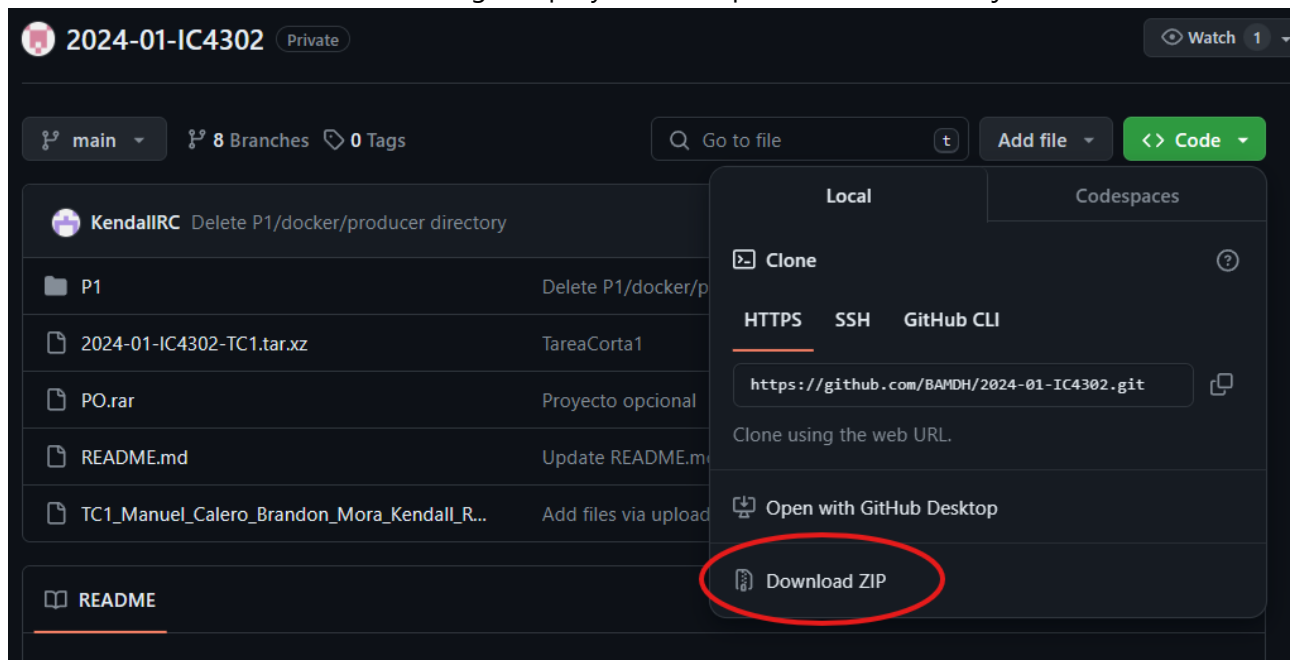
1. Se debe ingresar al repositorio de GitHub: <https://github.com/BAMDH/2024-01-IC4302>

2. Una vez dentro, se debe realizar lo siguiente:

- Seleccionar la opción verde llamada "Code"

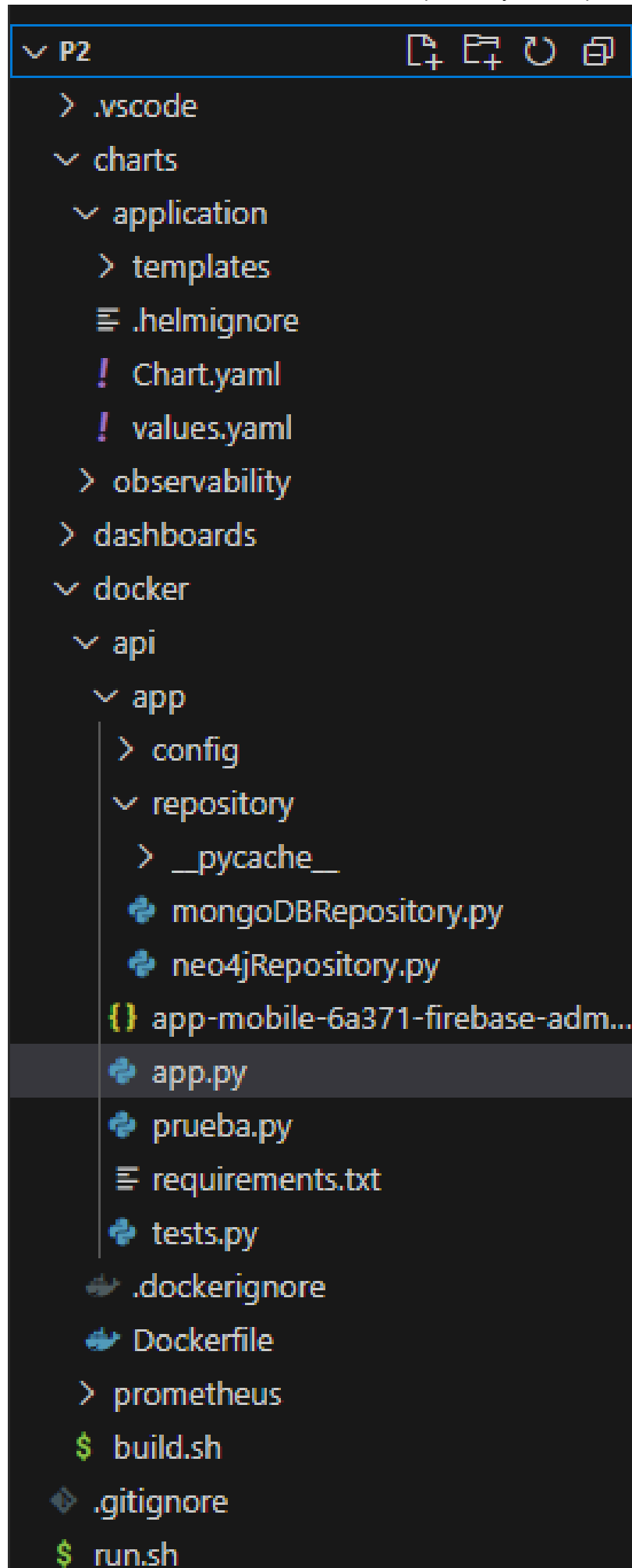


- Una vez seleccionado, se debe descargar el proyecto en la parte indicada con rojo



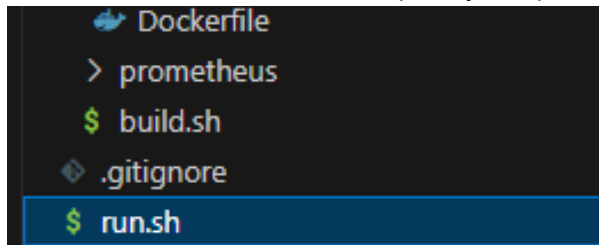
- Con esto se descargará el proyecto, el cual debería abrirse con el editor de código de referencia.

3. Cuando se abre el archivo, selecciona la carpeta P2 y debe aparecer lo siguiente:

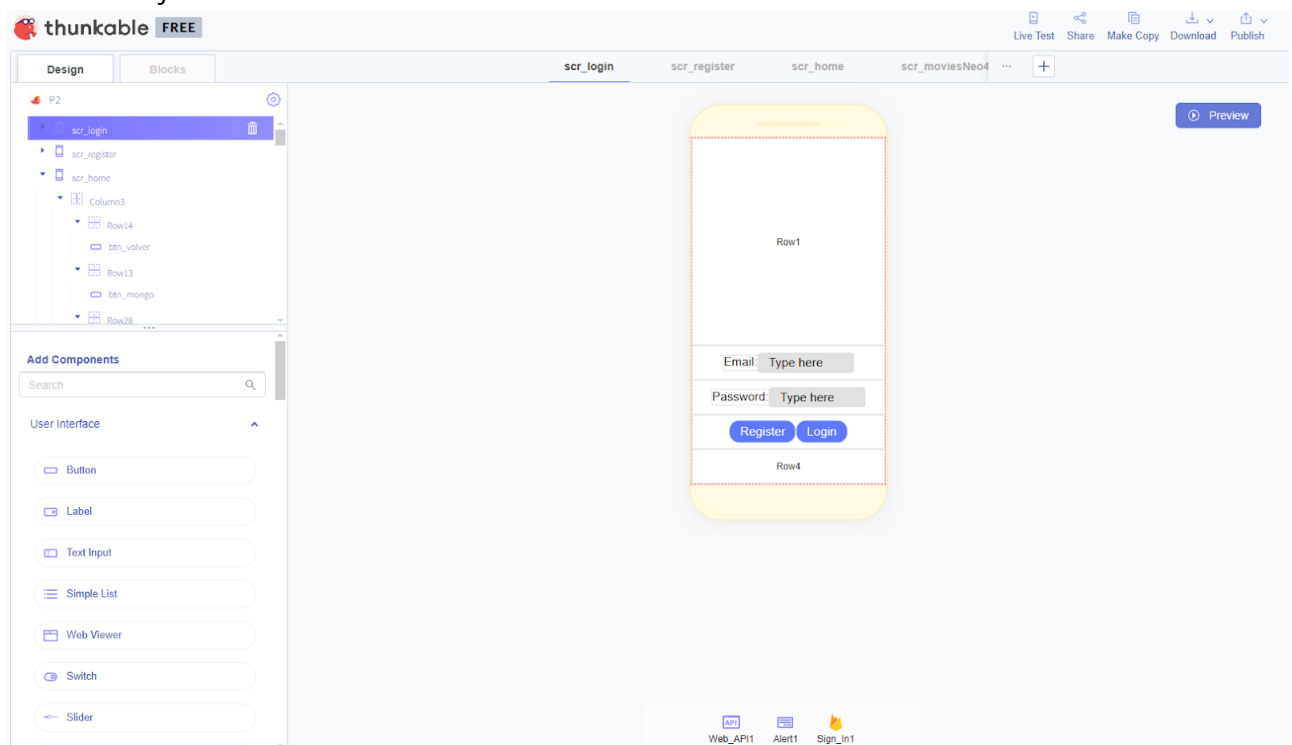


4. Se debe seleccionar el archivo llamado run.sh y ejecutar en la línea de comandos lo indicado dentro del archivo, pero antes se debe realiza:

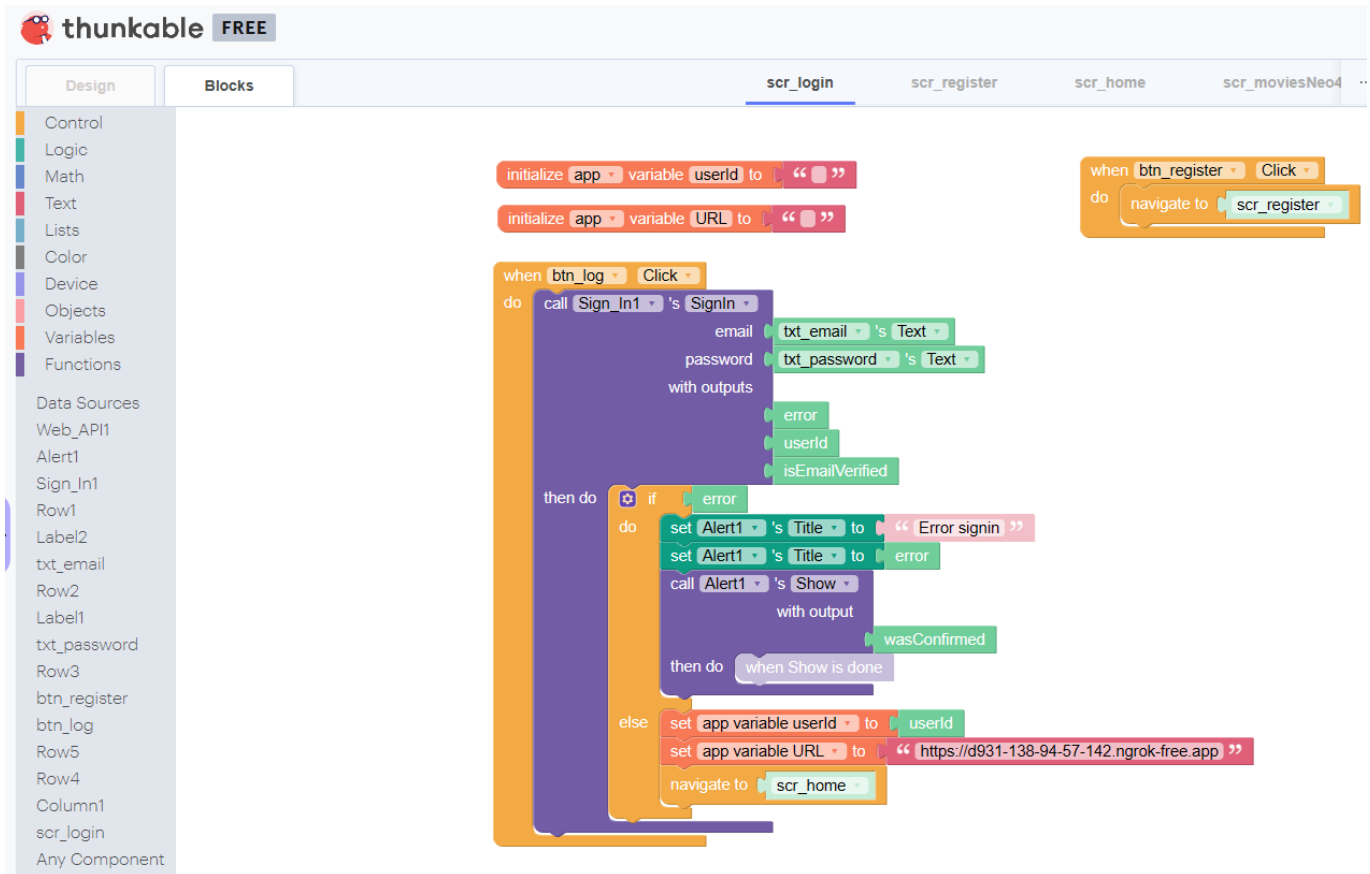
- Primero se debe tener abierto o en ejecución la aplicación de Docker
- Una vez en ejecución, se debe de ingresar en la línea de comandos (cmd) e ingresar a la ubicación de la carpeta P2.
- Cuando se esté dentro de la carpeta, ya se puede ejecutar lo que indica el run.sh



5. Una vez instalados todos los elementos, se debe ingresar a la página de thunkable, cuyo enlace a la interfaz de ejecución se indicó anteriormente.



6. Ahora debemos dirigirnos al apartado de "Blocks" y seleccionar la opción en círculo rojo, ahí sustituiremos ese link.



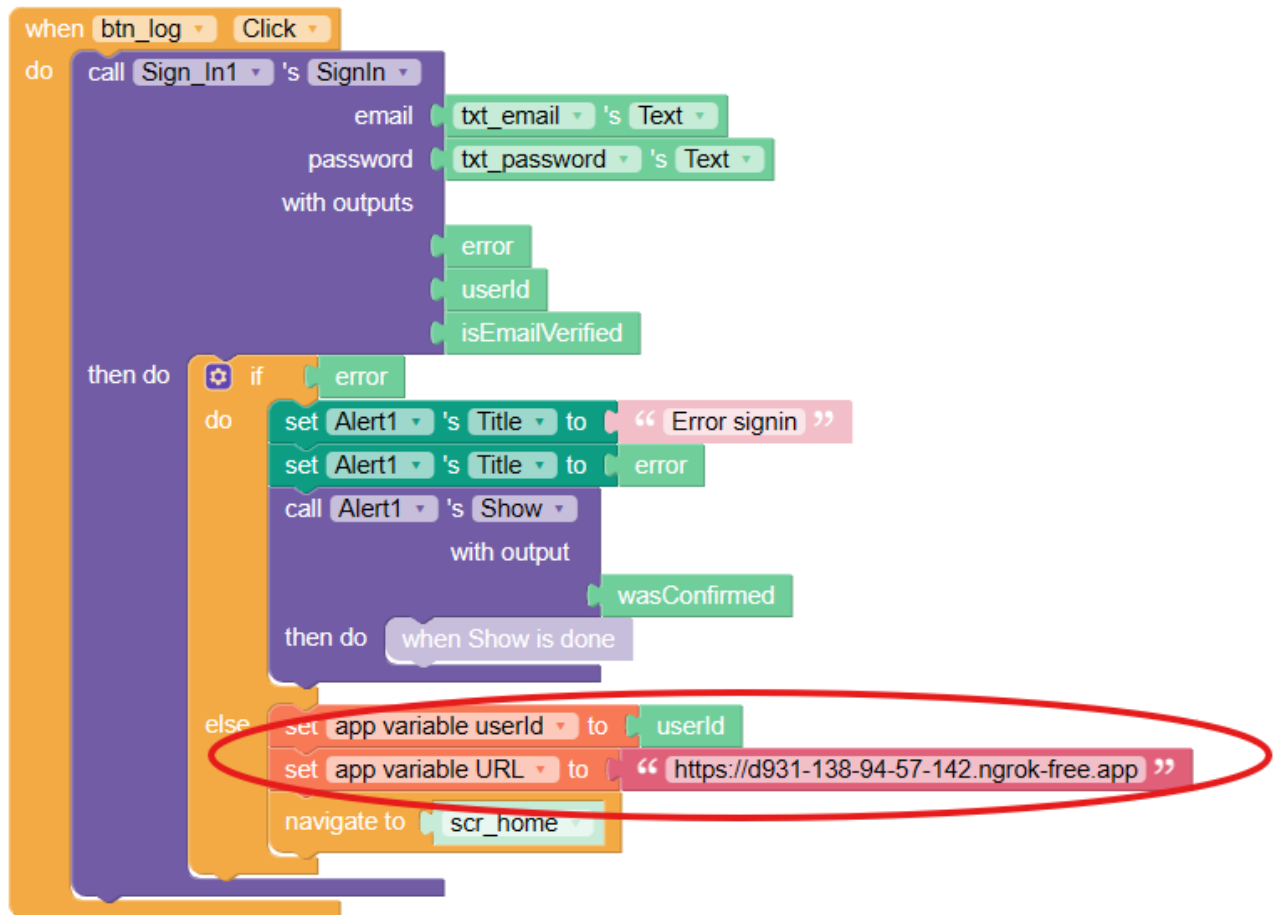
7. Dicho link será sustituido por usando ngrok, para ello ejecutamos el comando: `ngrok http http://localhost:5000` en la línea de comandos y seleccionamos lo marcado con rojo.

```
ngrok

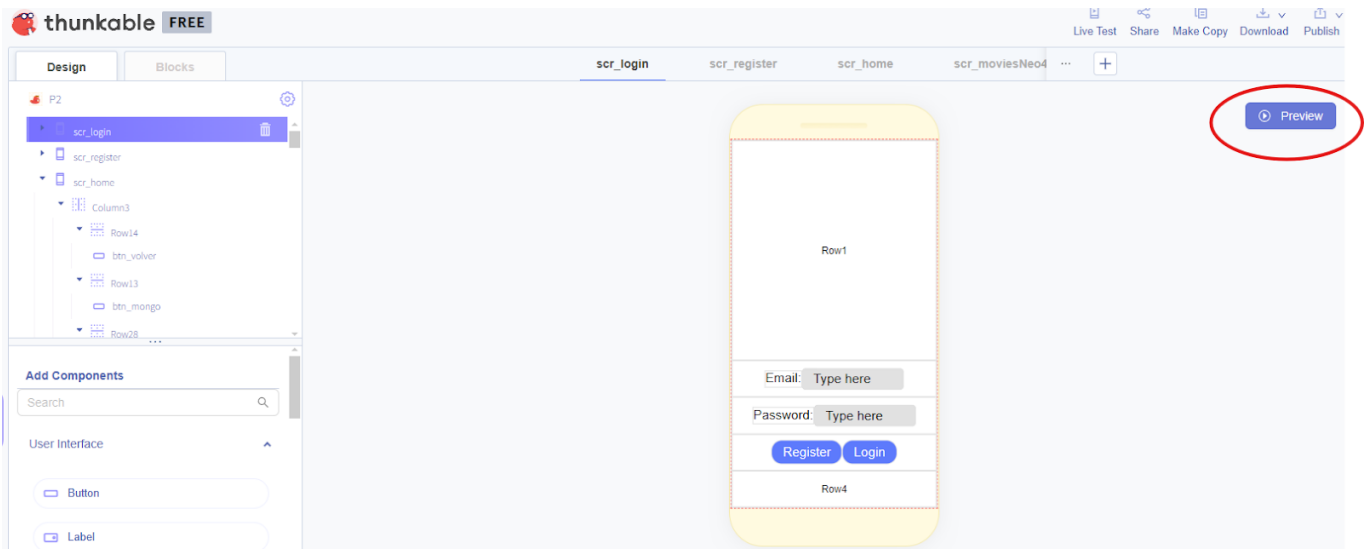
Try our new Traffic Inspector Dev Preview: https://ngrok.com/r/ti

Session Status      online
Account             manucalero76@gmail.com (Plan: Free)
Update              update available (version 3.10.0, Ctrl-U to update)
Version             3.9.0
Region              United States (us)
Web Interface        http://127.0.0.1:4040
Forwarding           https://c3d7-138-94-57-142.ngrok-free.app -> http://localhost:5000
```

8. Una vez copiado dicho link lo sustituimos en la opción antes descrita, recordando que el link debe terminar con `.app` sin ningún carácter extra:

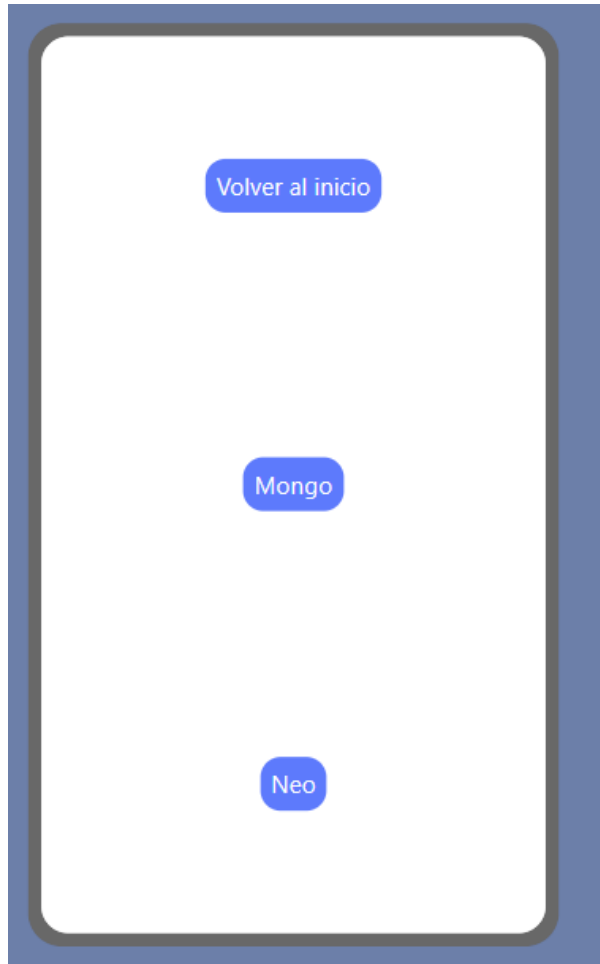


9. Ahora volvemos a "Desing" y ejecutamos la opción de "Preview" para poder interactuar con la interfaz.



10. Ahora estaremos interactuando con el menú principal, ahí tendremos dos opciones, registrar y login. En caso de tener cuenta, debe unicamente ingresar su email y su contraseña, en caso contrario debe crear un cuenta en la opción de register para poder ingresar.

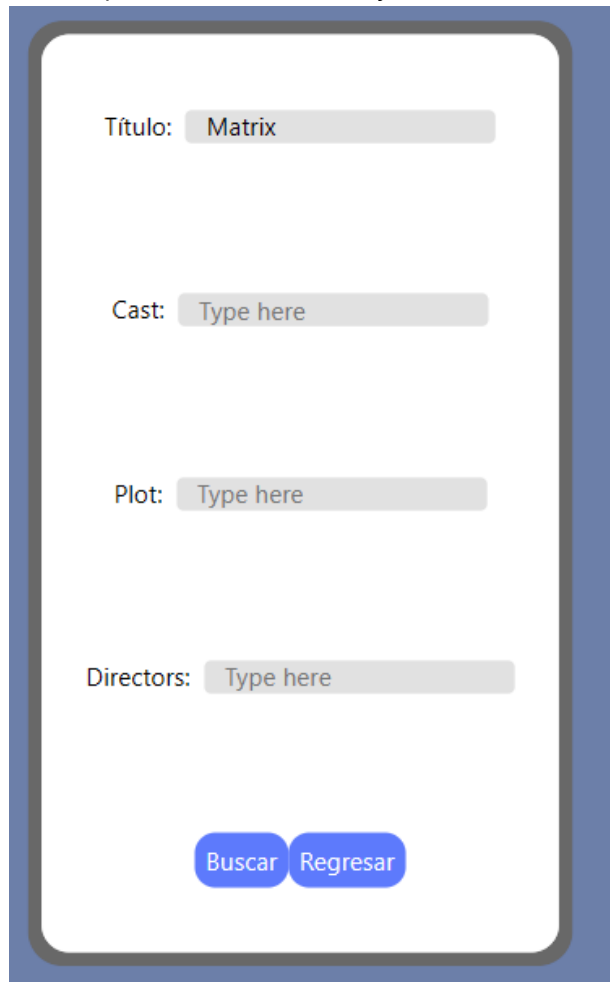
11. Una vez dentro, se despliega un menú que permite volver al inicio o seleccionar la base de datos que



desea consultar.

12. Cuando seleccione la base de datos a la que desea realizarle una consulta, se le desplegará otro menú, dicho menú contienen opciones en las cuales debe colocar el dato a consulta, como lo es el nombre de la película. En caso de no colocar nada no aparecerá información, por lo que es necesario de hacer. En

este ejemplo se filtra la película de Matrix. Cabe resaltar que el nombre de cast y directors deben ser



Título: Matrix

Cast: Type here

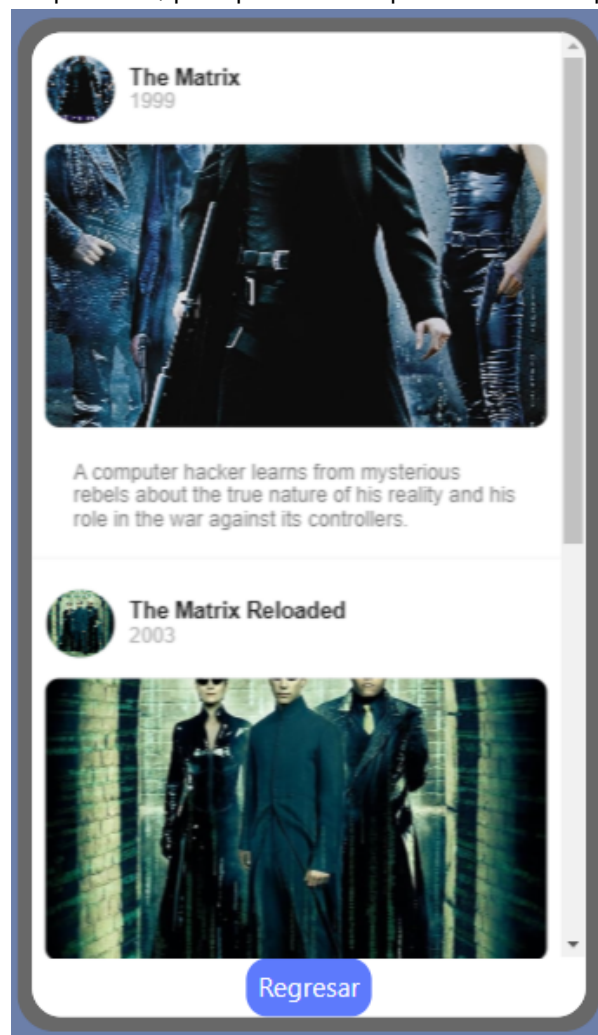
Plot: Type here

Directors: Type here

Buscar Regresar

exactos para poder realizar la búsqueda de ellos.

13. Una vez realizado, aparecerán las películas, para poder filtrar por medio de la película, debe precionar



sobre el nombre de la misma.

14. Cuando se selecciona se muestra una nueva interfaz, ahí seleccionaremos a la persona que deseamos filtrar y la opción de filtrar ya sea por actor o director, aclarando que si el switch está azul indica que se

busca como actor, mientras que gris busca por director.

The Matrix

A computer hacker learns from mysterious rebels about the true nature of his reality and his role in the war against its controllers.

Año: 1999

Cast: Keanu Reeves

Directores Andy Wachowski

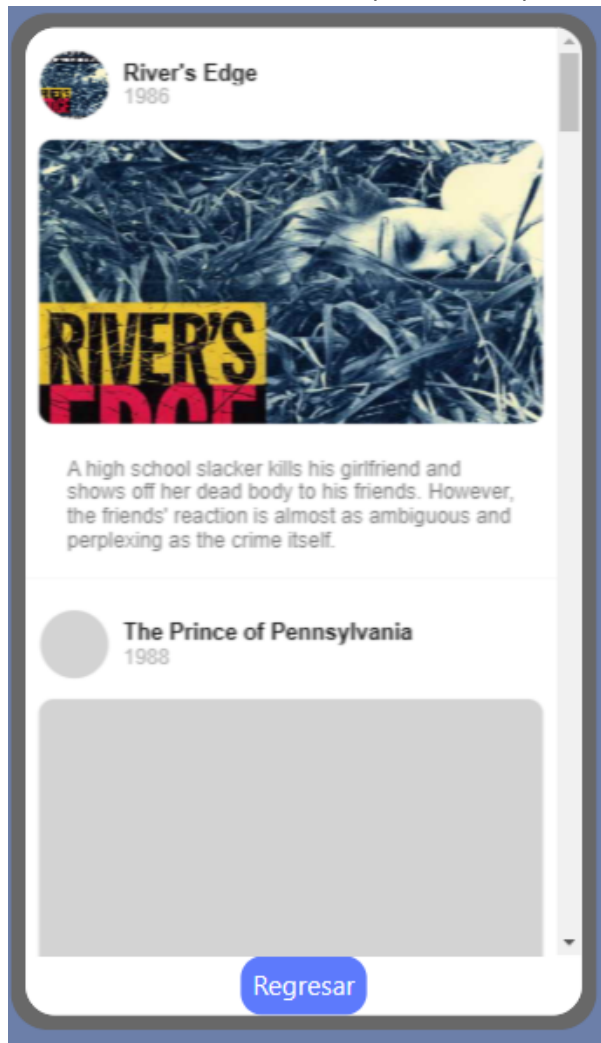
Género: Action,Sci-Fi

Nombre:

Director/Actor: ☐

Buscar

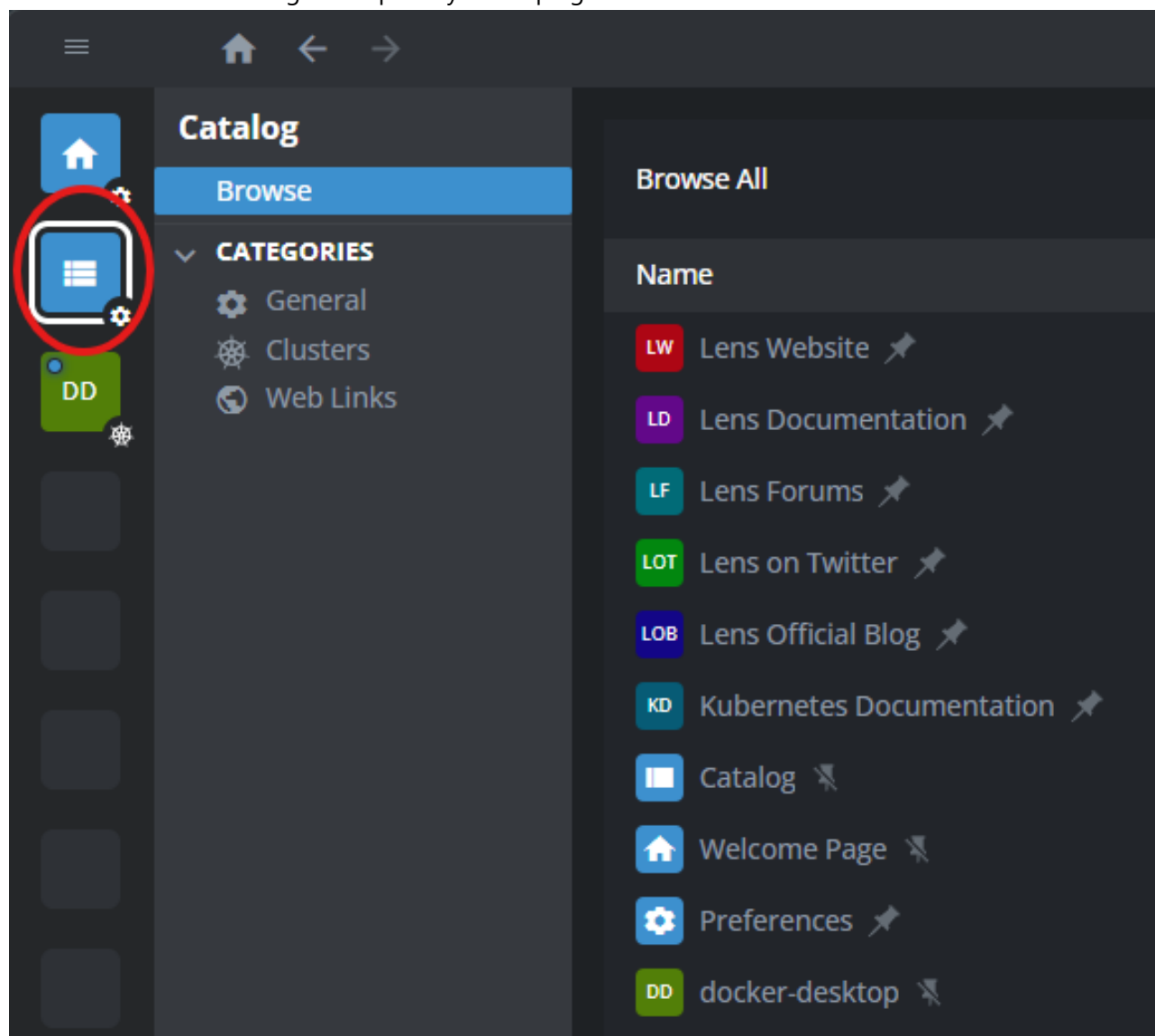
15. Cuando se filtra, en este caso a Keanu Reeves como actor, saldrán todas las películas disponibles en la



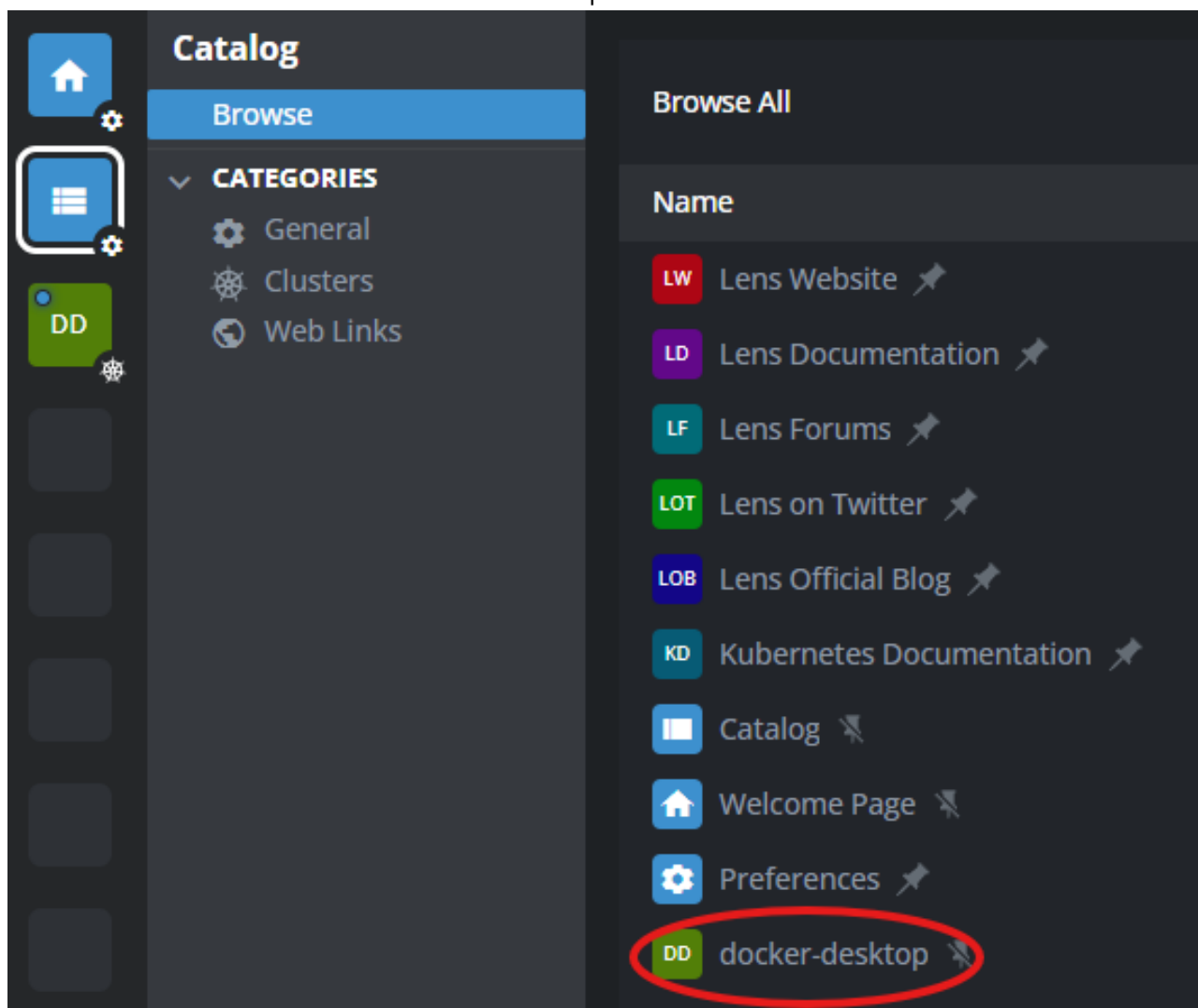
base de datos que cumplan dichos requisitos.

16. Con esto finalizarían las instrucciones de consulta por medio de esta interfaz, lo mismo aplica para Neo, con la diferencia de que contienen datos distintos.
17. Ahora solo queda observar el comportamiento del sistema por medio de observabilidad, pero para ello necesitaremos ingresar a Docker por medio de Lens:

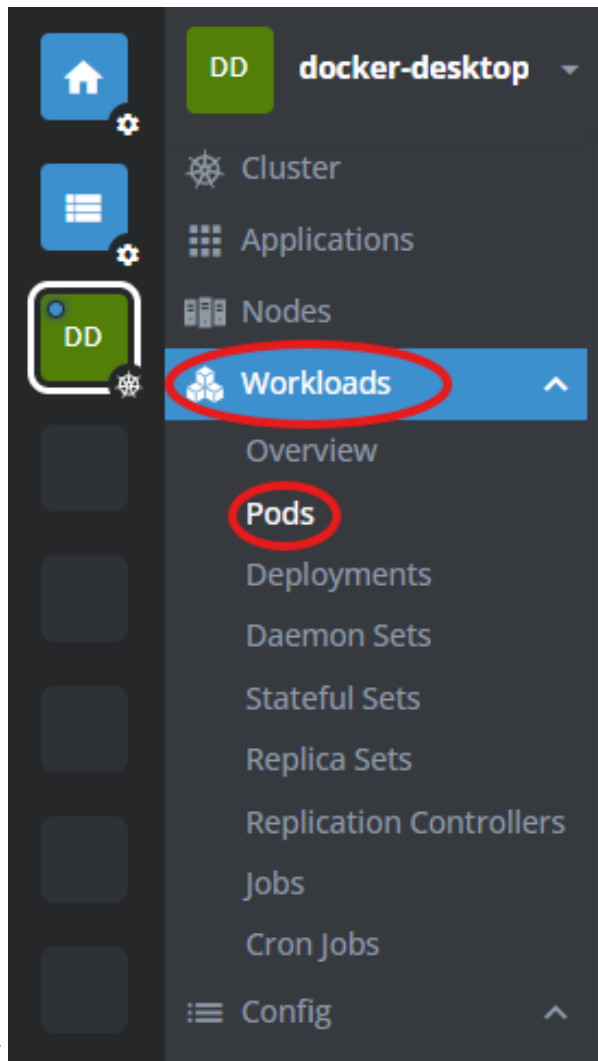
- Se debe seleccionar la segunda opción y se desplegarán varios elementos



- Ahí se debe seleccionar el llamado "docker-desktop"

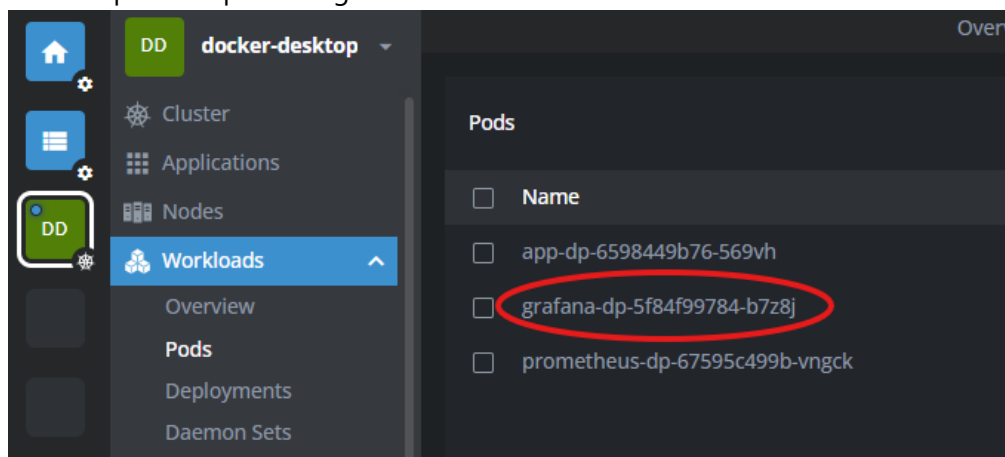


- Con eso se abrirá el Docker en Lens y se mostrarán diversos de sus elementos, se debe seleccionar el llamado "Workloads", el cual desplegará una serie de elementos, de los cuales se seleccionará el

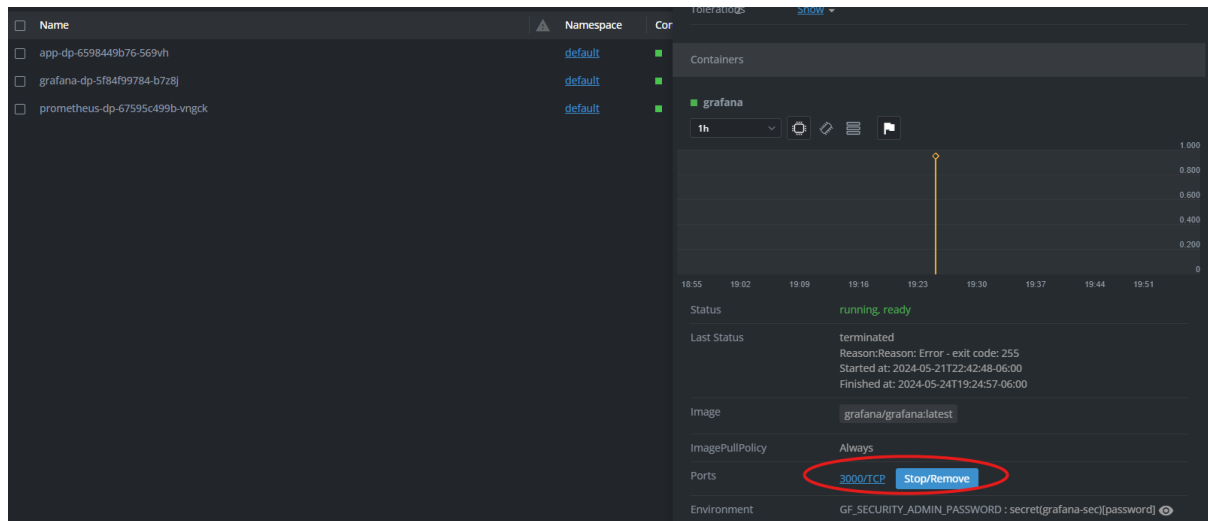


llamado "Pods"

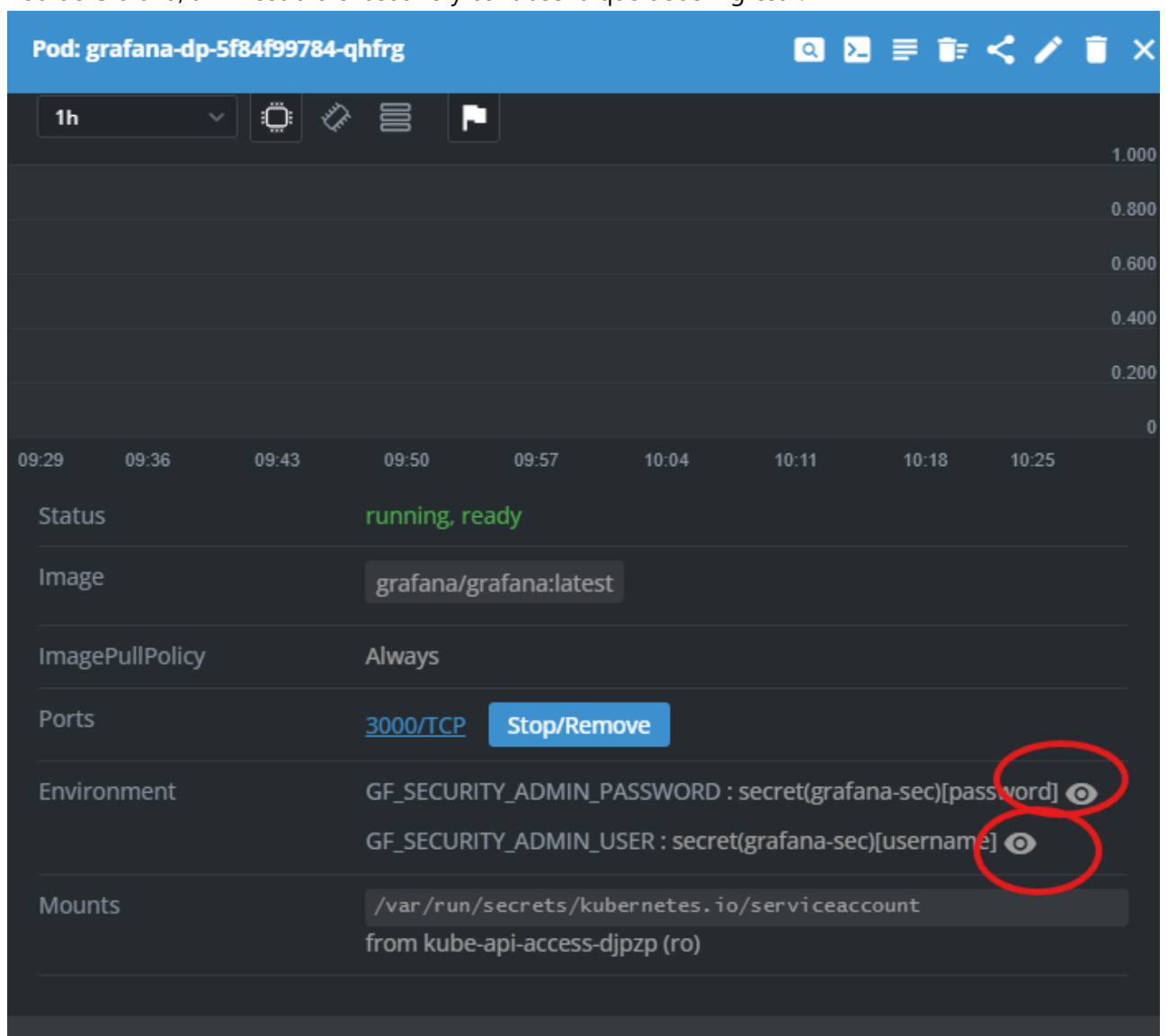
18. Una vez ingresado en Docker y seleccionada la opción de Pods, se debe buscar el elemento cuyo nombre posea la palabra "grafana"



19. Una vez seleccionado se mostrará un menú a la derecha, el cual bajaremos hasta hallar la opción de puertos y seleccionar el número que sale en celeste y colocar manualmente el puerto 3000.

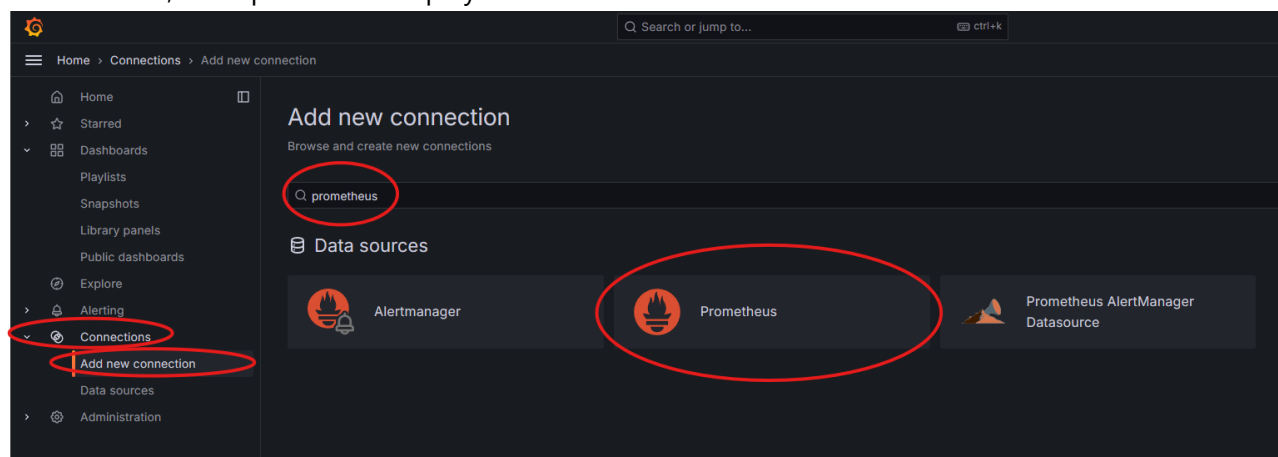


20. Una vez dentro, se solicita usuario y contraseña, para obtenerlo presione en los ojos que salen en el Pod de Grafana, ahí mostrará el usuario y contraseña que debe ingresar.



21. Al ingresar el usuario y contraseña se desplegará el menú del Grafana, ahora debe ir a la parte de la izquierda y seleccionar "Connections", ahí dentro debe darle a "Add new connection". Una vez

seleccionado, debe presionar la lupa y buscar "Prometheus".



22. Una vez seleccionado, hace falta configurarlo, por lo que debe de introducir el puerto de Prometheus y crear la conexión.

The screenshot shows the Prometheus configuration page in Grafana. At the top, the Prometheus logo and name are displayed. Below, the 'Type' is set to 'Prometheus'. There are two tabs: 'Settings' (active) and 'Dashboards'. Under 'Settings', the 'Name' is 'prometheus' and the 'Default' toggle is off. A message states: 'Before you can use the Prometheus data source, you must configure it below or in the config file. Fields marked with * are required'. The 'Connection' section has a 'Prometheus server URL *' field containing 'http://prometheus-sv:9090', which is circled in red. Below this is the 'Exemplars' section with an '+ Add' button. A green success message says 'Successfully queried the Prometheus API.' with instructions to build a dashboard or use the explore view. At the bottom, there are 'Delete' and 'Save & test' buttons, with the latter circled in red.

prometheus

Type: Prometheus

Settings Dashboards

Name ⓘ prometheus Default ☐

Before you can use the Prometheus data source, you must configure it below or in the config file.
Fields marked with * are required

Connection

Prometheus server URL * ⓘ http://prometheus-sv:9090

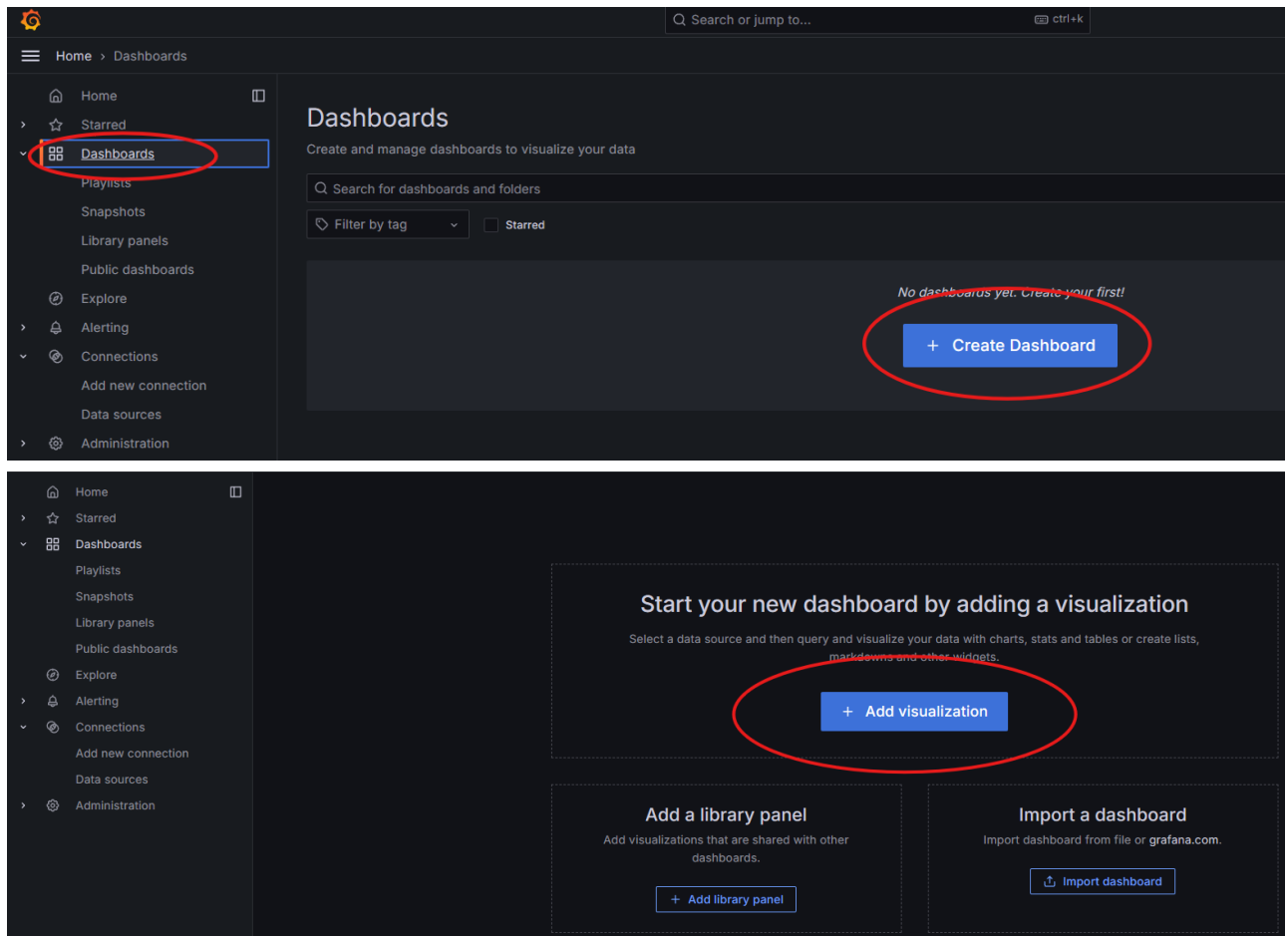
Exemplars

+ Add

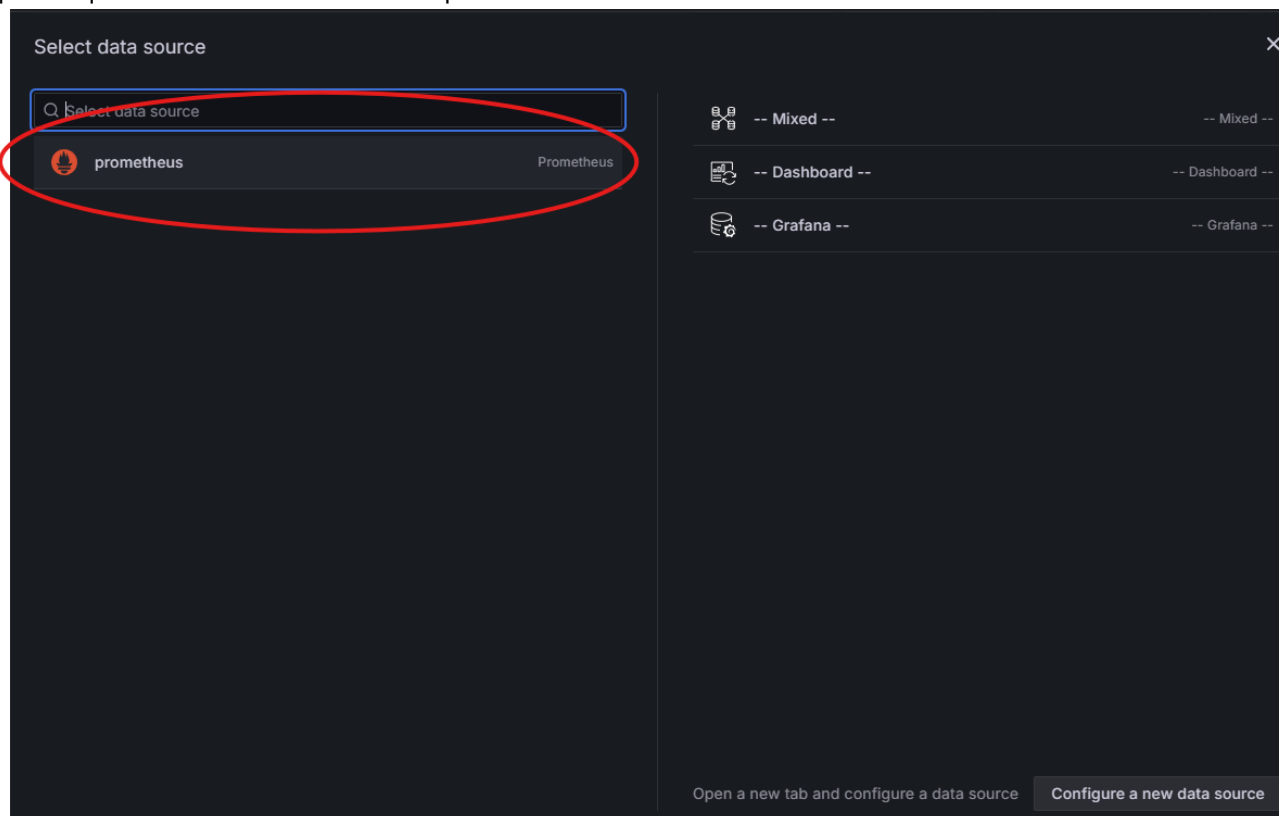
✓ Successfully queried the Prometheus API.
Next, you can start to visualize data by [building a dashboard](#), or by querying data in the [Explore view](#).

Delete Save & test

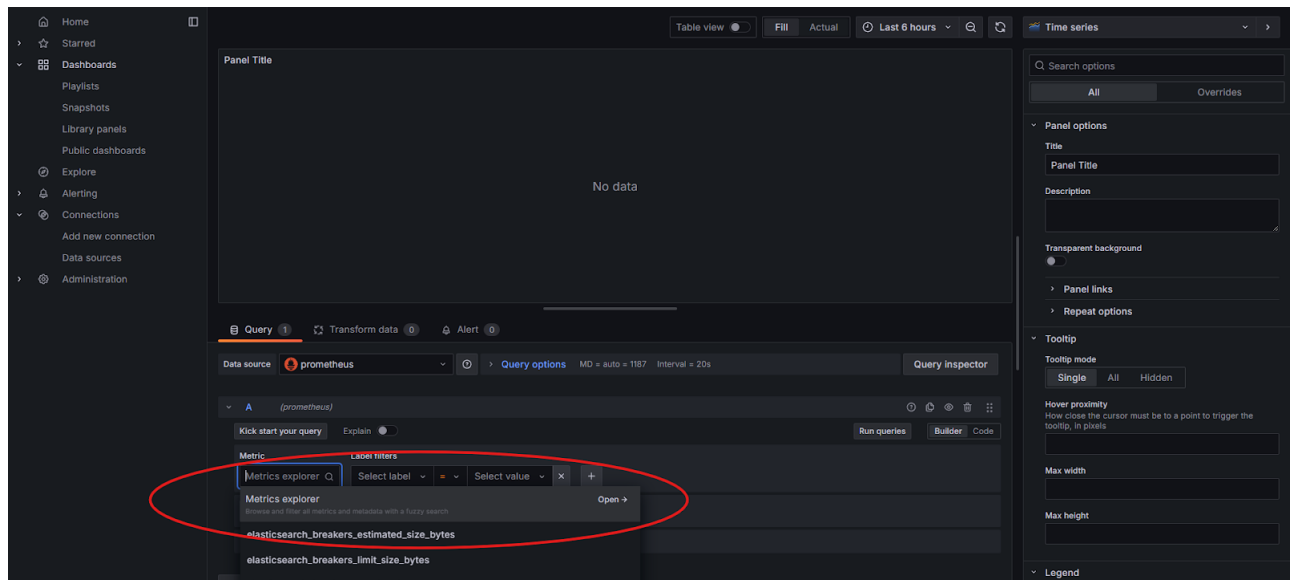
23. Una vez configurado, nos iremos a la parte de "Dashboards" en la izquierda y darle a "Create Dashboard". Ahí dentro debe darle a "Add visualization".



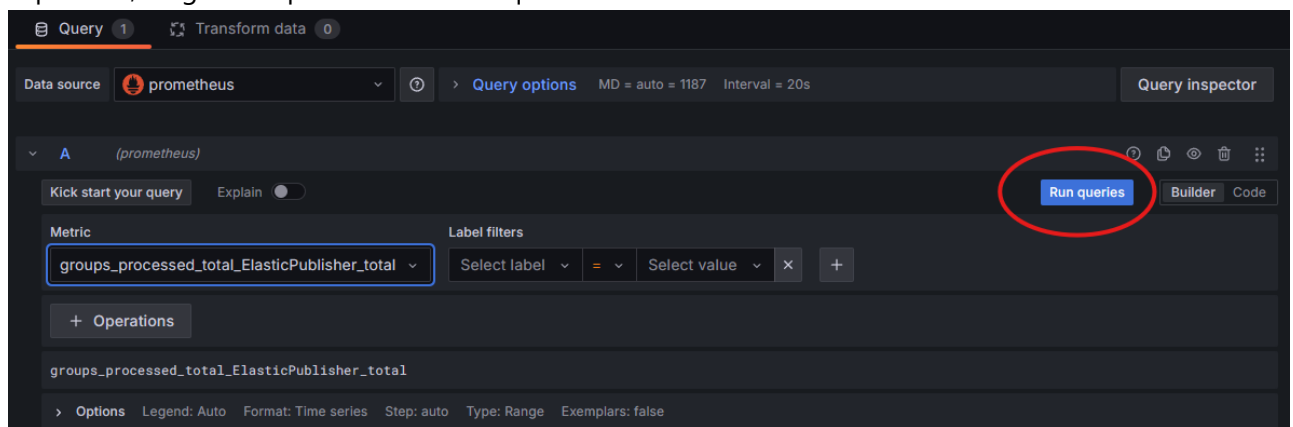
24. Una vez hecho, se despliegan varias opciones, pero de momento nos interesa únicamente Prometheus, por lo que solo seleccionamos esa opción.



25. Ahora se despliega un nuevo menú, aquí ya seleccionamos las métricas que deseamos ver, para ello seleccionamos la opción de "Metrics explorer".



26. Cuando estemos dentro ya podemos seleccionar entre las muchas gráficas que hay o buscar una en específico, luego sería presionar el "Run queries".



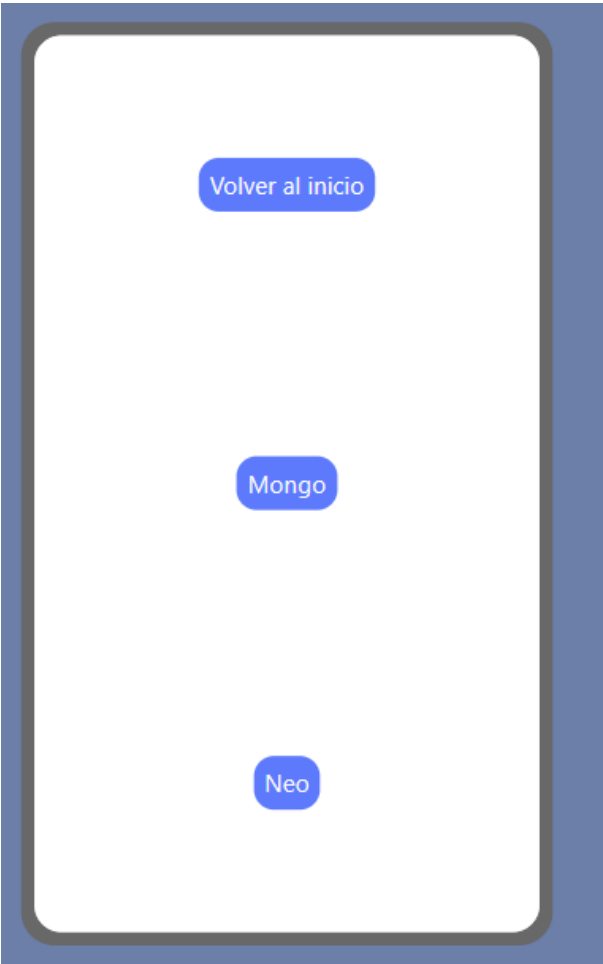
27. Ya con eso hemos ejecutado todos los procesos disponibles en el proyecto.

Pruebas realizadas con pasos

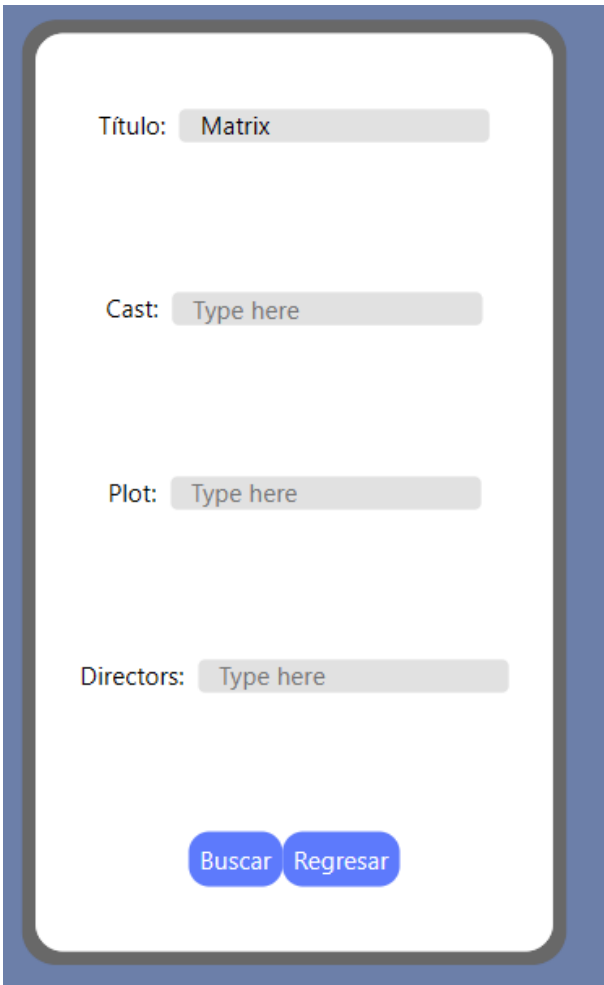
Prueba 1

Para esta prueba se hará uso de la misma interfaz de Thunkable para consultar datos. Para ello realizamos:

- Primero iniciamos sesión.

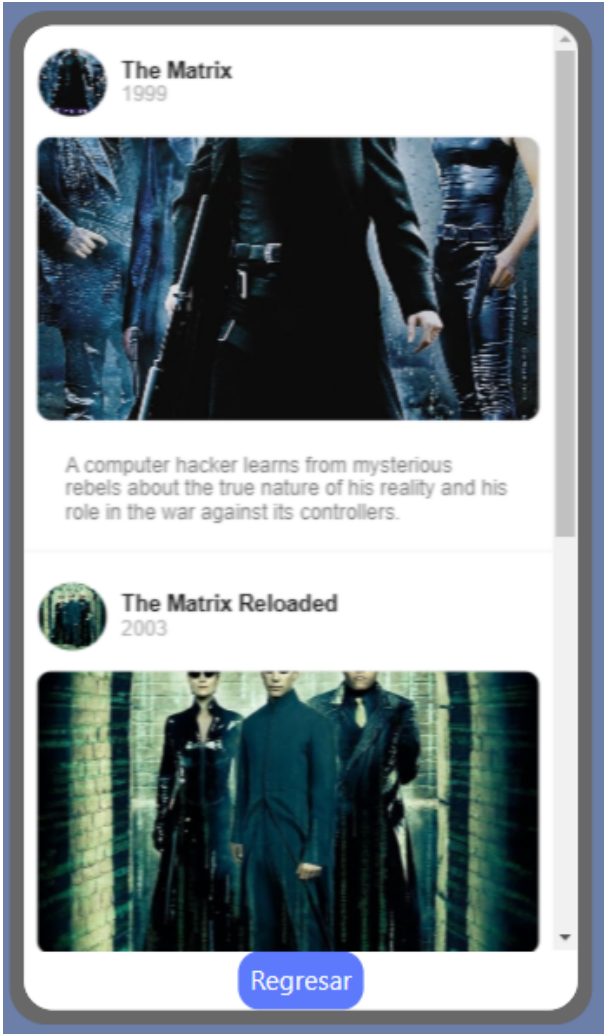


- Una vez dentro seleccionaremos mongo.



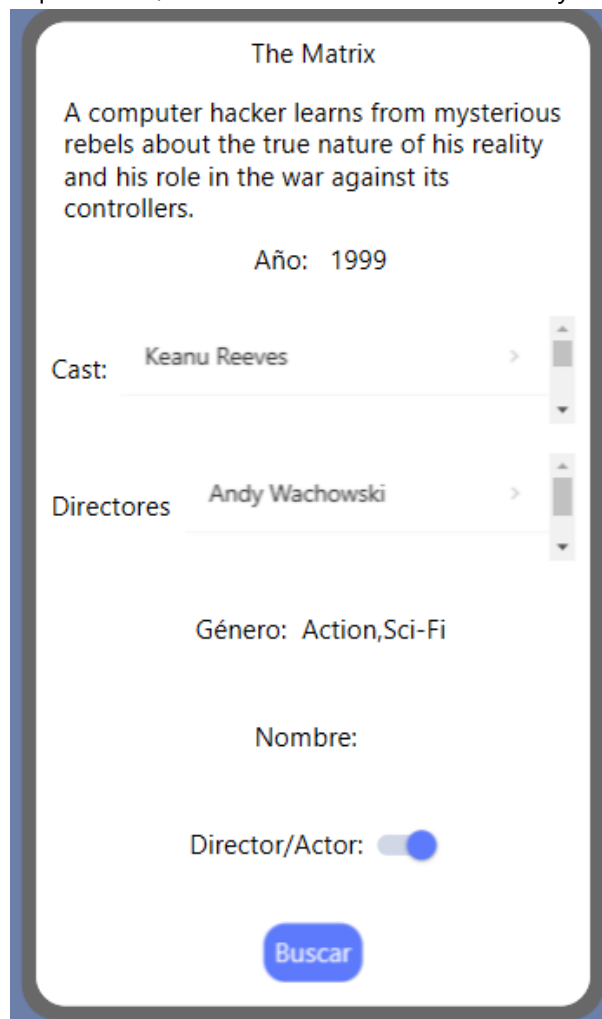
- Buscaremos una película, en este caso Matrix.

- Con la información que se despliega filtraremos por una película, para ello seleccionamos el nombre de



la primera.

- Aquí dentro, seleccionamos a Keanu Reeves y lo filtramos como actor.



The Matrix

A computer hacker learns from mysterious rebels about the true nature of his reality and his role in the war against its controllers.

Año: 1999

Cast: Keanu Reeves

Directores: Andy Wachowski

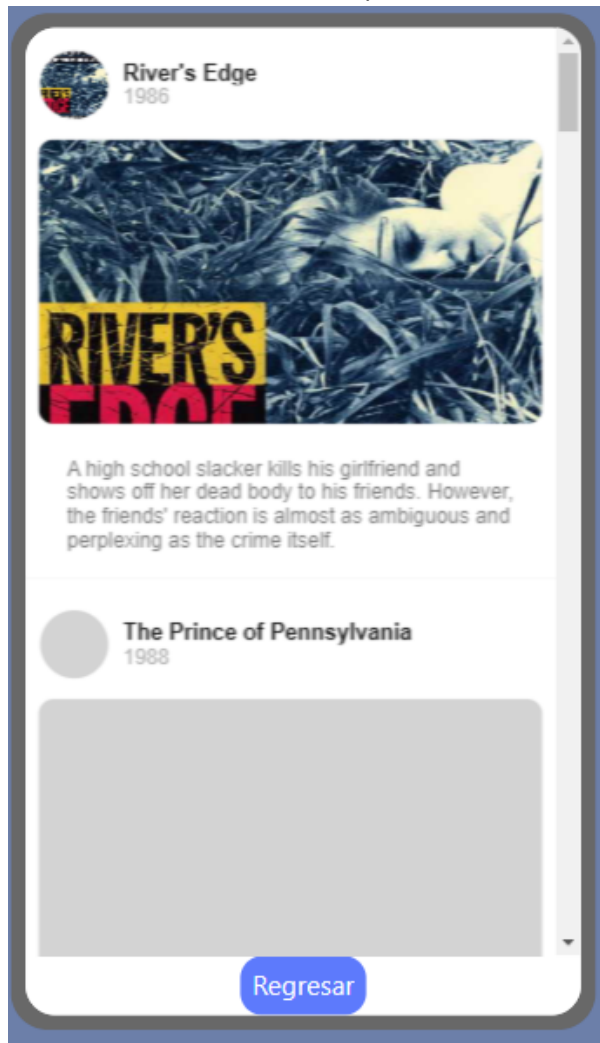
Género: Action,Sci-Fi

Nombre:

Director/Actor: ☒

Buscar

- Ahora se muestran todas las películas donde él es actor.



Prueba 2:

Realizando el mismo proceso anterior, pero haciendo uso de la base de datos de Neo.

- Primero iniciamos sesión.

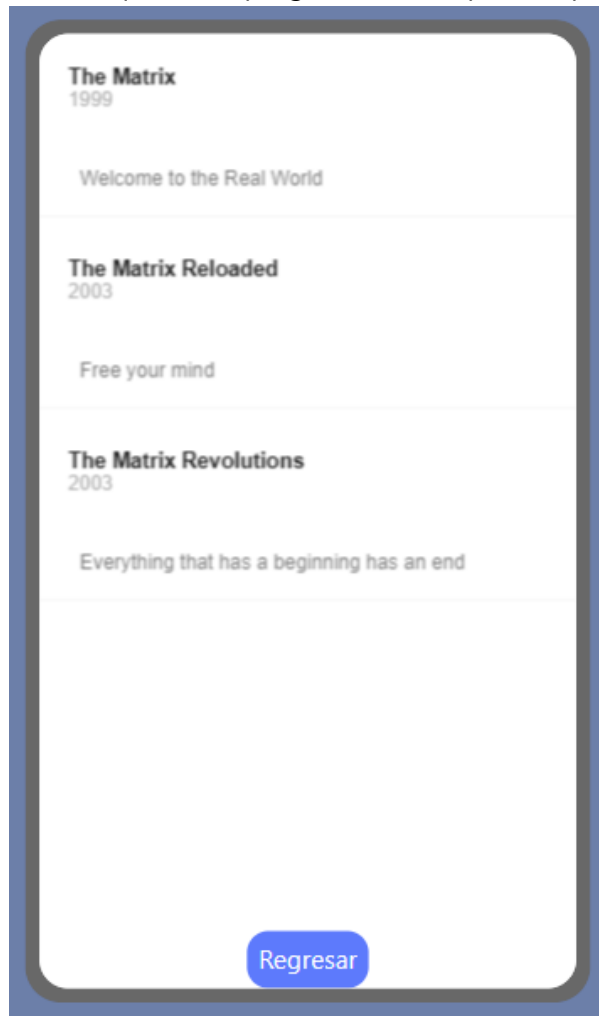


- Una vez dentro seleccionaremos Neo.

A mobile application screen with a white background and a blue border. It contains a form with four input fields, each with a label and a placeholder text. The labels are 'Título:', 'Cast:', 'Plot:', and 'Directors:'. The placeholder texts are 'Matrix', 'Type here', 'Type here', and 'Type here' respectively. At the bottom, there are two blue buttons with white text: 'Buscar' and 'Regresar'.

- Buscaremos una película, en este caso Matrix.

- Con la información que se despliega filtraremos por una película, para ello seleccionamos el nombre de



la primera.

- Aquí dentro, seleccionamos a Keanu Reeves y lo filtramos como actor.

The Matrix

Welcome to the Real World

Cast:

Emil Eifrem

Hugo Weaving

Directores:

Lana Wachowski

Lilly Wachowski

Año:

1999

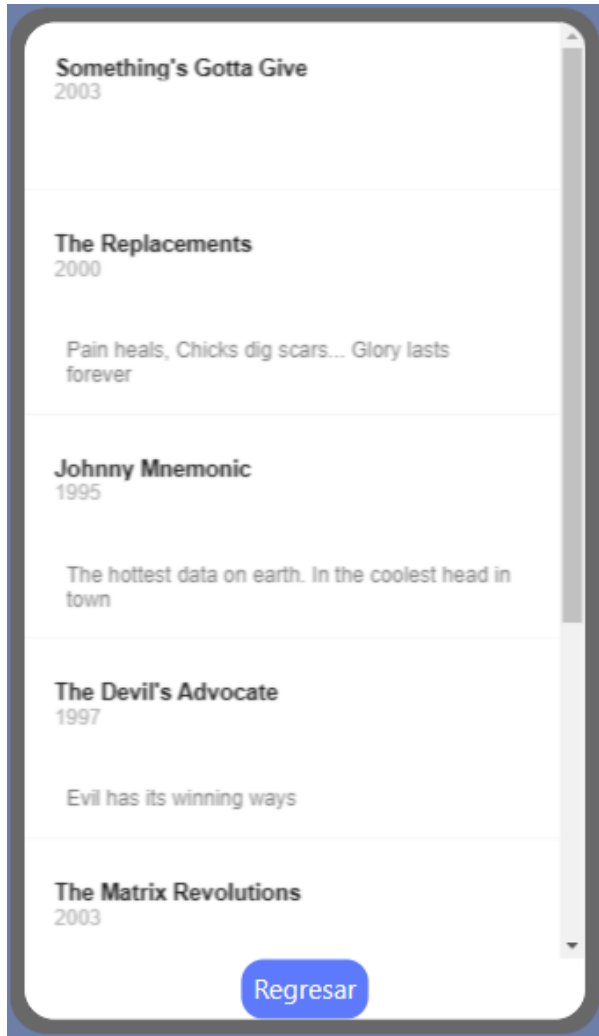
Nombre:

Director/Actor:

☒

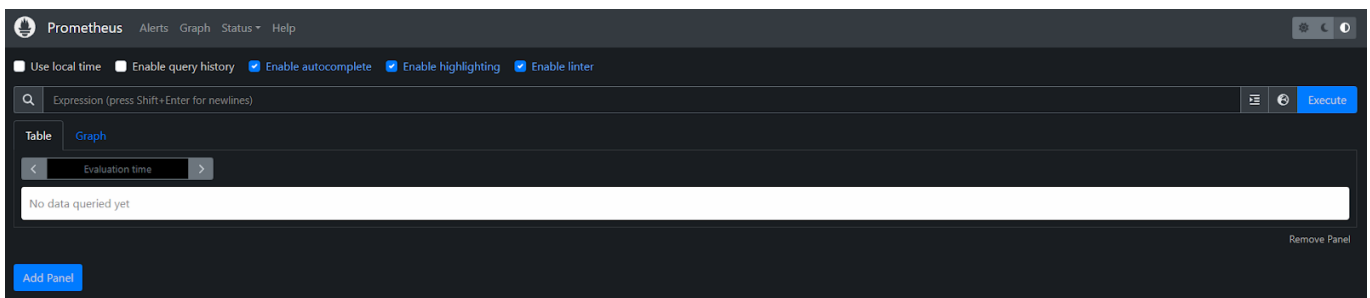
Buscar

- Ahora se muestran todas las películas donde él es actor.

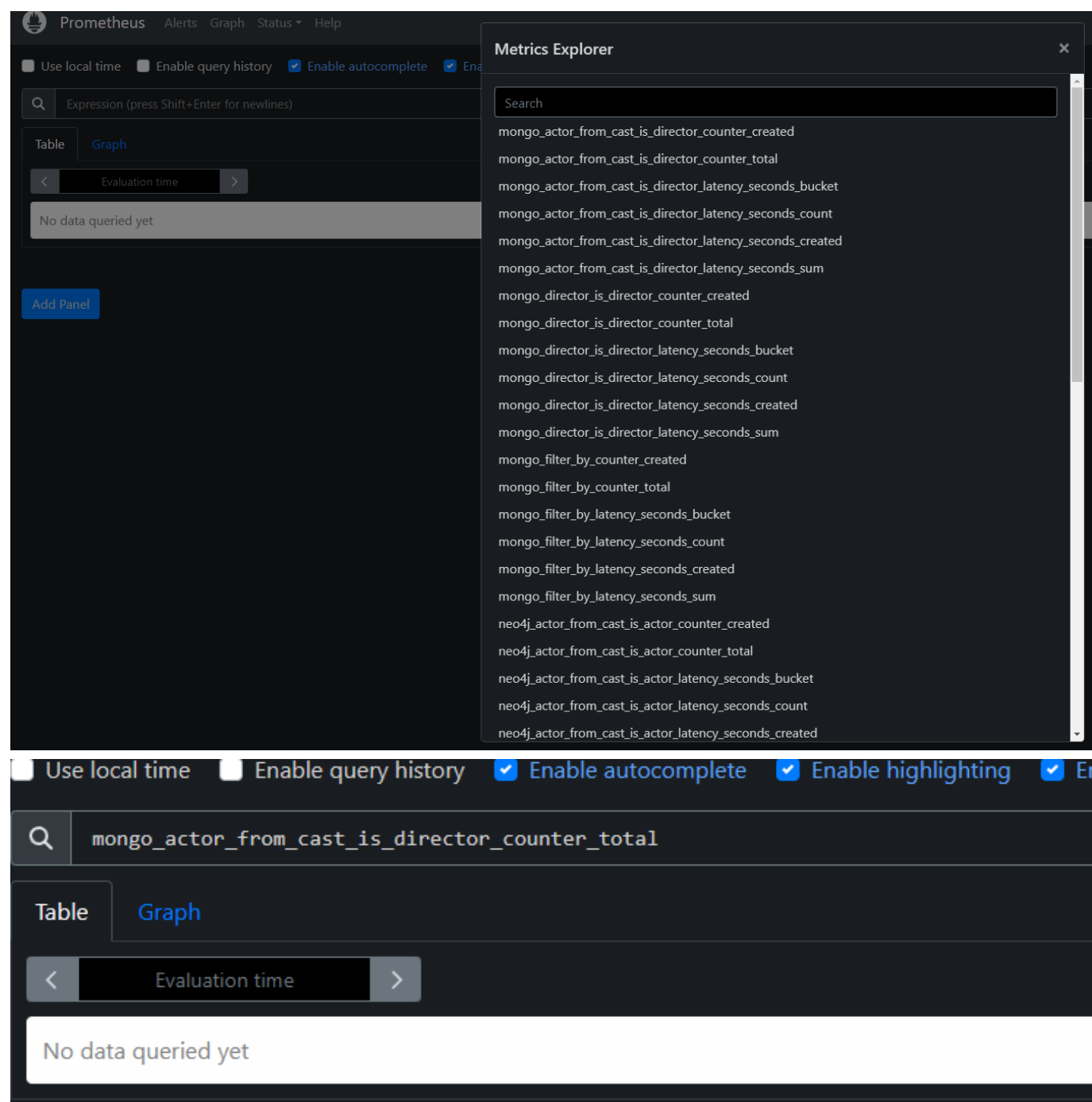


Prueba 3:

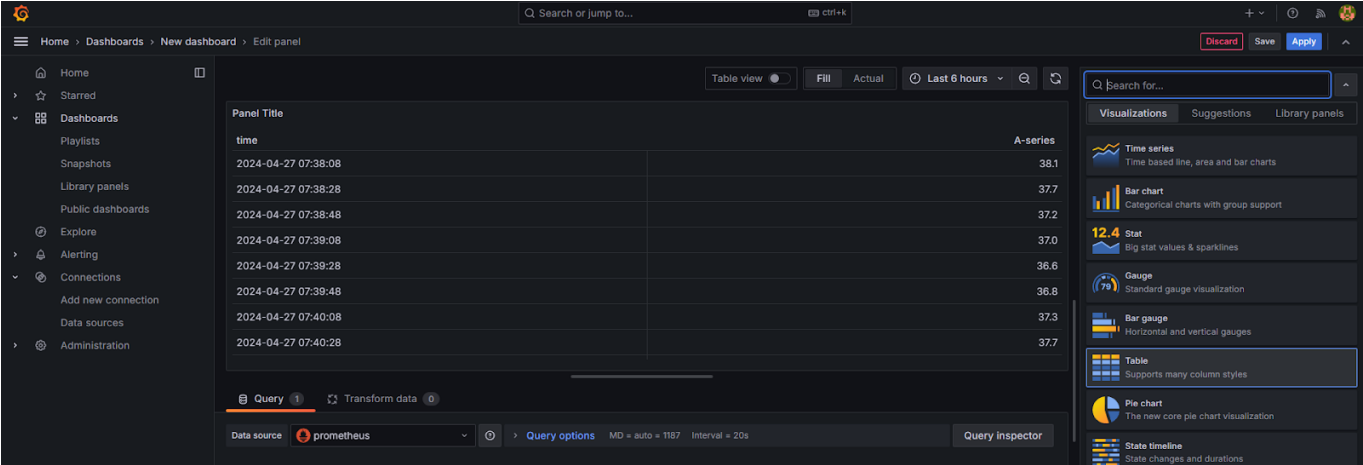
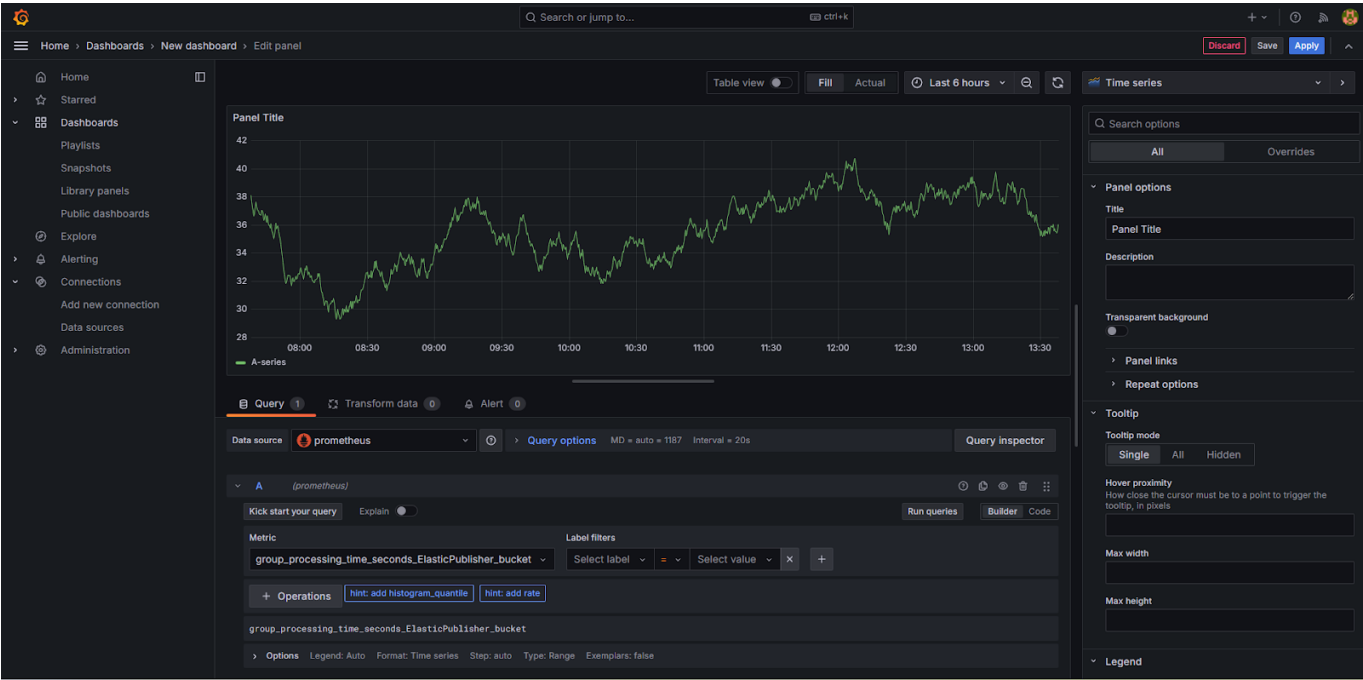
Esta prueba consiste en poner a práctica los diferentes componente de observabilidad. Como primer instancia, ingresamos desde Lens en el pod cuyo nombre contenga la palabra "Prometheus" y seleccionamos su puerto, el método de ingreso a jobs ya se indicó en las instrucciones de ejecución del proyecto.



Una vez dentro, solo seleccionamos el menú de Prometheus y buscamos la métrica a consultar, en este caso se selecciona una de mongo.

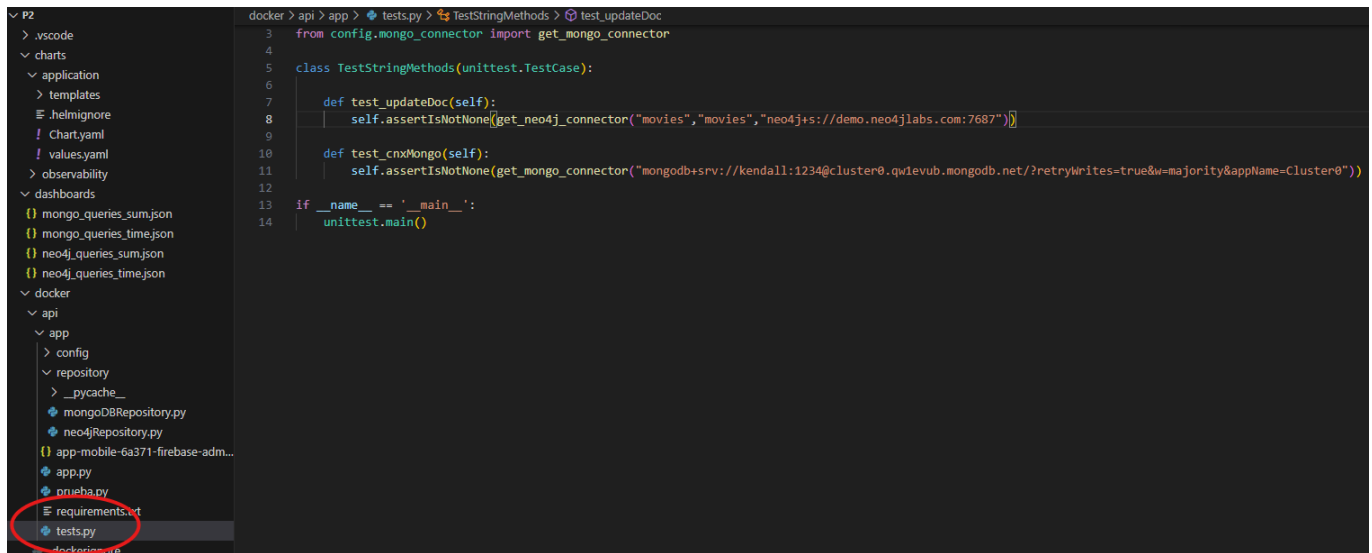


Este es el proceso dentro de Prometheus, ahora podemos consultarlo desde Grafana. Una vez dentro seleccionamos la misma métrica que consultamos desde Prometheus, con la diferencia que podemos seleccionar la forma en la cual observar la gráfica, como se indica en el círculo rojo.



Resultados pruebas unitarias

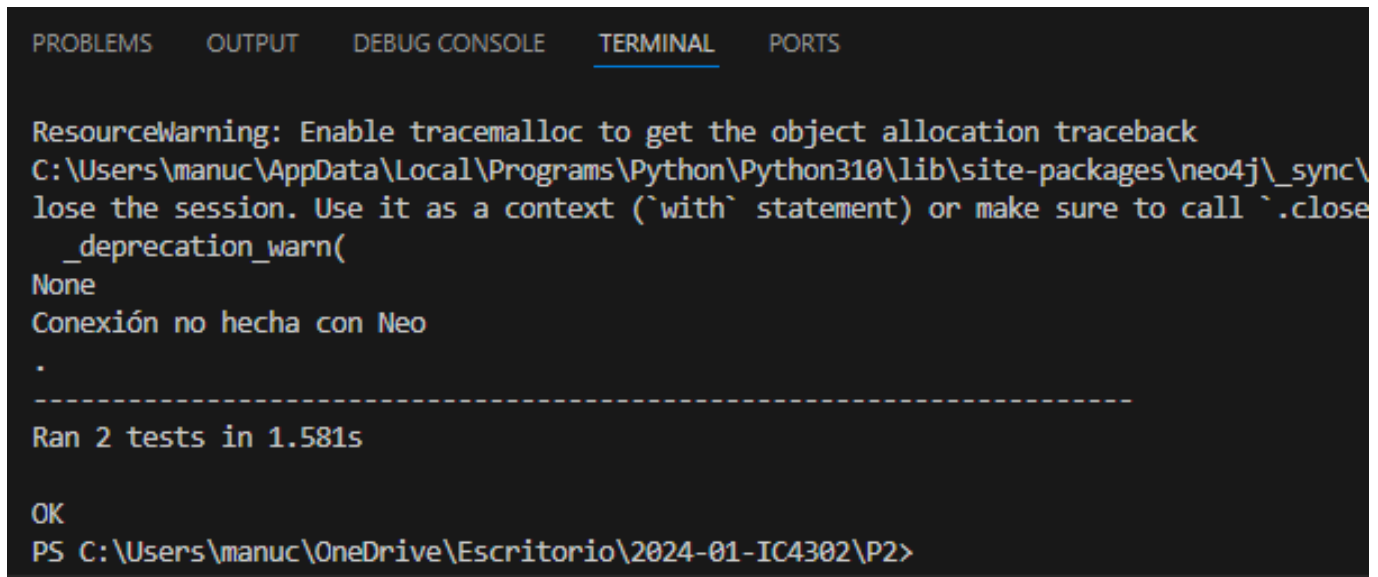
Para las pruebas unitarias, se verificó la conexión con las bases de datos, para ello se ejecutó el test.py ubicado dentro del proyecto.



The image shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'charts', 'application', 'templates', 'helmignore', 'Chart.yaml', 'values.yaml', 'observability', 'dashboards', and 'docker'. Under 'docker', there is a folder 'api' which contains a folder 'app'. Inside 'app', there is a file 'tests.py' which is highlighted with a red circle. The code editor shows the content of 'tests.py'.

```
docker > api > app > tests.py > TestStringMethods > test_updateDoc
3 from config.mongo_connector import get_mongo_connector
4
5 class TestStringMethods(unittest.TestCase):
6
7     def test_updateDoc(self):
8         self.assertIsNotNone(get_neo4j_connector("movies", "movies", "neo4j+s://demo.neo4jlabs.com:7687"))
9
10    def test_cnxMongo(self):
11        self.assertIsNotNone(get_mongo_connector("mongodb+srv://kendall:1234@cluster0.qwievub.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0"))
12
13    if __name__ == '__main__':
14        unittest.main()
```

Una vez ejecutado el archivo .py, este desplegará si la conexión con las bases de datos fue exitosa. La verificación de dicha conexión se concluye como cierta si el resultado de finalización es un OK, en este caso se comprueba que las conexiones son correctas.



The image shows a terminal window with the following output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\manuc\AppData\Local\Programs\Python\Python310\lib\site-packages\neo4j\_sync\
lose the session. Use it as a context (`with` statement) or make sure to call `.close
_deprecation_warn(
None
Conexión no hecha con Neo
.
-----
Ran 2 tests in 1.581s
OK
PS C:\Users\manuc\OneDrive\Escritorio\2024-01-IC4302\P2>
```

El caso anterior es cuando ambas funcionan, pero si se hace un cambio que no permita la conexión, se desplegará un mensaje indicativo del fallo, como se muestra a continuación realizando un cambio en el URL de la conexión a Mongo.

```
tests.py X
docker > api > app > tests.py > TestStringMethods > test_cnxMongo
1  import unittest
2  from config.neo4j_connector import get_neo4j_connector
3  from config.mongo_connector import get_mongo_connector
4
5  class TestStringMethods(unittest.TestCase):
6
7      def test_updateDoc(self):
8          self.assertIsNotNone(get_neo4j_connector("movies","movies"),
9
10     def test_cnxMongo(self):
11         self.assertIsNotNone(get_mongo_connector("mongodb+srv://ken
12
13 if __name__ == '__main__':
14     unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
FAIL: test_cnxMongo (__main__.TestStringMethods)
-----
Traceback (most recent call last):
  File "c:\Users\manuc\OneDrive\Escritorio\2024-01-IC4302\P2\docker\api\app\tests
    self.assertIsNotNone(get_mongo_connector("mongodb+srv://kendall:123@cluster0.
AssertionError: unexpectedly None
-----
Ran 2 tests in 1.685s

FAILED (failures=1)
PS C:\Users\manuc\OneDrive\Escritorio\2024-01-IC4302\P2> 
```

Ahora, si se realiza un cambio en el URL de la conexión a Neo, como sustituir su puerto 7687 por 7686, podemos observar que aquí que se produce el fallo de conexión.

```
tests.py X
docker > api > app > tests.py > TestStringMethods > test_updateDoc
1 import unittest
2 from config.neo4j_connector import get_neo4j_connector
3 from config.mongo_connector import get_mongo_connector
4
5 class TestStringMethods(unittest.TestCase):
6
7     def test_updateDoc(self):
8         self.assertIsNotNone(get_neo4j_connector(["movies", "movies", "neo4j+s://demo.neo4jlabs.com:7686"]))
9
10    def test_cnxMongo(self):
11        self.assertIsNotNone(get_mongo_connector("mongodb+srv://kendall:1234@cluster0.qw1evub.mongodb.net
12
13 if __name__ == '__main__':
14     unittest.main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

FAIL: test_updateDoc (__main__.TestStringMethods)
-----
Traceback (most recent call last):
  File "c:\Users\manuc\OneDrive\Escritorio\2024-01-IC4302\P2\docker\api\app\tests.py", line 8, in test_updateDoc
    self.assertIsNotNone(get_neo4j_connector(["movies", "movies", "neo4j+s://demo.neo4jlabs.com:7686"]))
AssertionError: unexpectedly None
-----
Ran 2 tests in 22.566s

FAILED (failures=1)
PS C:\Users\manuc\OneDrive\Escritorio\2024-01-IC4302\P2>
```

Recomendaciones y conclusiones

Recomendaciones

1. Se recomienda utilizar un entorno de desarrollo como Visual Studio Code, ya que es práctico y permitirá ejecutar de manera sencilla cada una de las partes del programa.
2. Debe seguir los pasos indicados en el método de ejecución del programa, ya que realizarlos en otro orden puede ocasionar que ciertos elementos o el proyecto como tal no funcione.
3. La versión de python debe ser superior a la 3.10 debido a que posee un proceder distinto en diferentes instancias del código, esto principalmente si desea crear las imágenes usted mismo.
4. En caso de realizar modificaciones variadas, como agregar o quitar print para un pod y observarlo en el logs de Lens, debe ejecutar el app.py utilizando comandos de docker. En este caso, se recomienda limpiar el Dockerhub en línea, ya que en ocasiones daba error al crear imágenes y la solución que se encontró fue limpiarlo.
5. Si desea ejecutar el proyecto de manera local, debe considerar sustituir las variables que hacen conexiones. En este caso, se utilizan NodePort que son diferentes para cada pod, así que debe buscar

dentro del Service en Lens el de cada pod, además considere que cada que hace una instalación nueva usando Helm, estos puertos pueden variar.

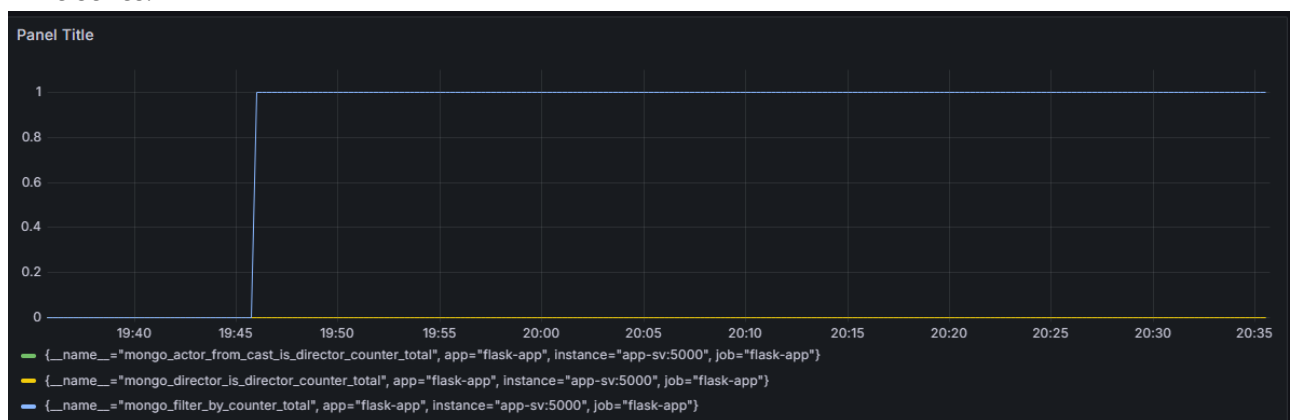
6. En caso de desinstalar el application se recomienda borrar los elementos del índice jobs y groups, esto porque puede presentar errores ya que pueden quedar valores intermedios de la anterior ejecución. Los comandos para ello, se encuentran dentro del proyecto en el archivo comandosElastic.txt.
7. En la carpeta dashboard dentro del proyecto, al querer ejecutar alguno de los ejemplos dentro del Grafana, van a presentar un error de ejecución, esto se debe a que cada archivo JSON que genera el Grafana contiene un id especial por cada inicio de sesión, así que se recomienda sustituir el id de cada JSON de ejemplo por el suyo y así pueda consultar los ejemplos.
8. Si consulta el Prometheus, puede en ocasiones no observar algunas métricas en específico, esto ocurre principalmente a la hora de instalar el application o el observability, dicho problema ocurre porque aún el proceso no ha llegado a Prometheus, por lo que se recomienda que, si ocurre dicho problema, se actualice la página pasados unos 30 segundos para así ver todas las métricas.
9. Si desinstala el observability debe considerar que al ejecutarlo nuevamente y abrir el Grafana, podrá observar que no tendrá el dashboard aún teniendo la conexión a Prometheus. El problema que ocurre es que se desvincula el service, por lo que deberá introducir el service nuevamente dentro del Grafana para así tener nuevamente el dashboard.
10. Debe realizar el proceso de sustitución de link en Thunkable cada vez que ejecute nuevamente el ngrok, ya que sino se realiza, la aplicación del Trunkable puede no mostrar datos debido a que está consultando un link viejo.

Conclusiones

Para las conclusiones, se utilizarán las gráficas en formato JSON ubicadas dentro del proyecto, en este caso serán 4.

1. mongo_queries_sum:

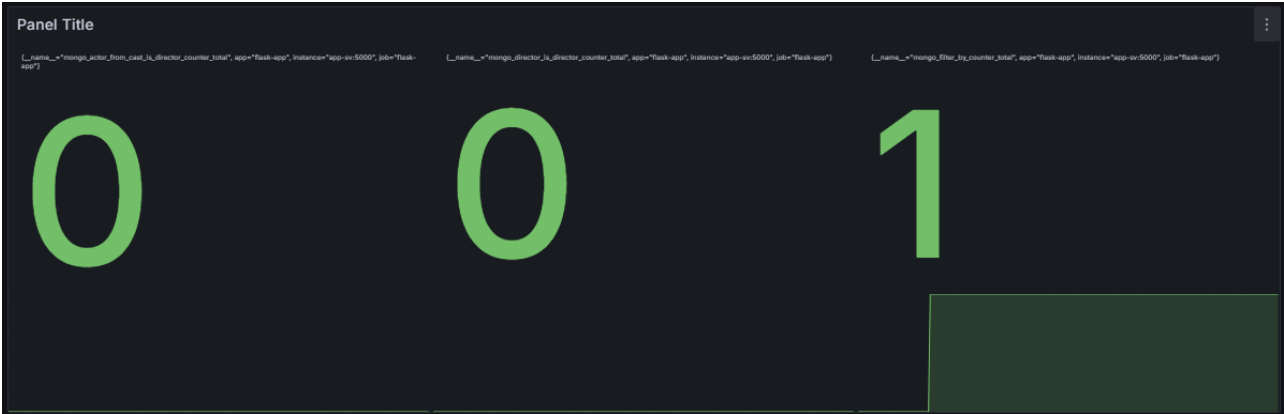
- Time series:



- Bar chart:



- Stat:

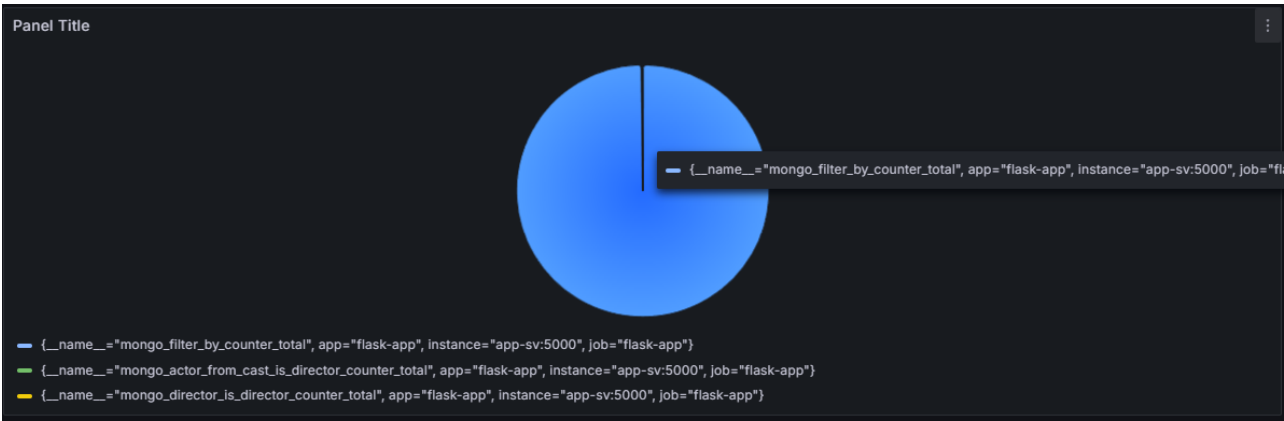


- Table:

Panel Title	
Time	{__name__="mongo_actor_from_cast_is_director_counter_total", app="flask-app", ir
2024-05-24 19:35:30	0
2024-05-24 19:35:45	0
2024-05-24 19:36:00	0
2024-05-24 19:36:15	0
2024-05-24 19:36:30	0
2024-05-24 19:36:45	0
2024-05-24 19:37:00	0

{__name__="mongo_actor_from_cast_is_director_counter_total", app="flask-app", instance="app-sv:5000", job="flask-app"}

- Pie chart:



En este caso, las gráficas dejan en claro que se hizo una única consulta de búsqueda, principalmente en el área de filtrado. Aquí la que muestra la información con mayor claridad es la de Stat, ya que nos indica directamente el número de queries realizados en mongo.

2. mongo_queries_time:

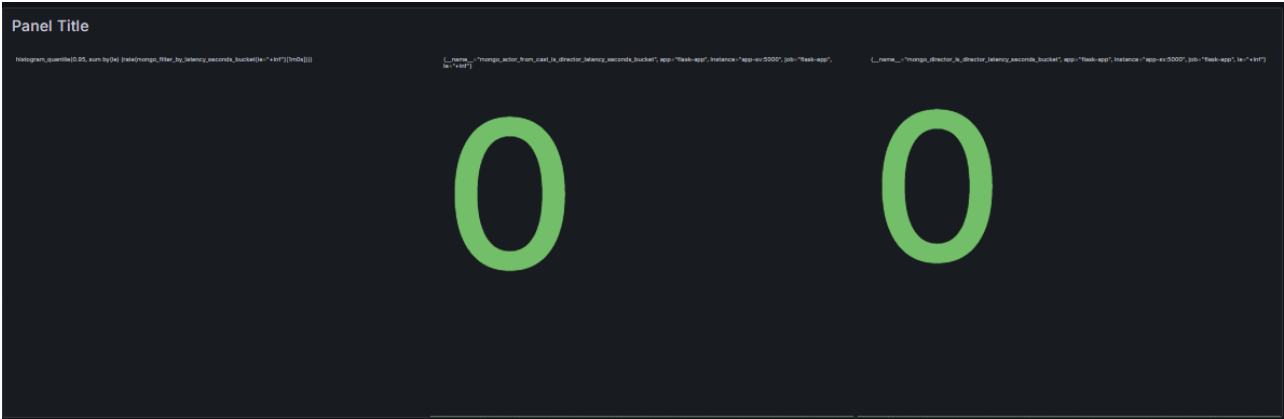
- Time series:



- Bar chart:



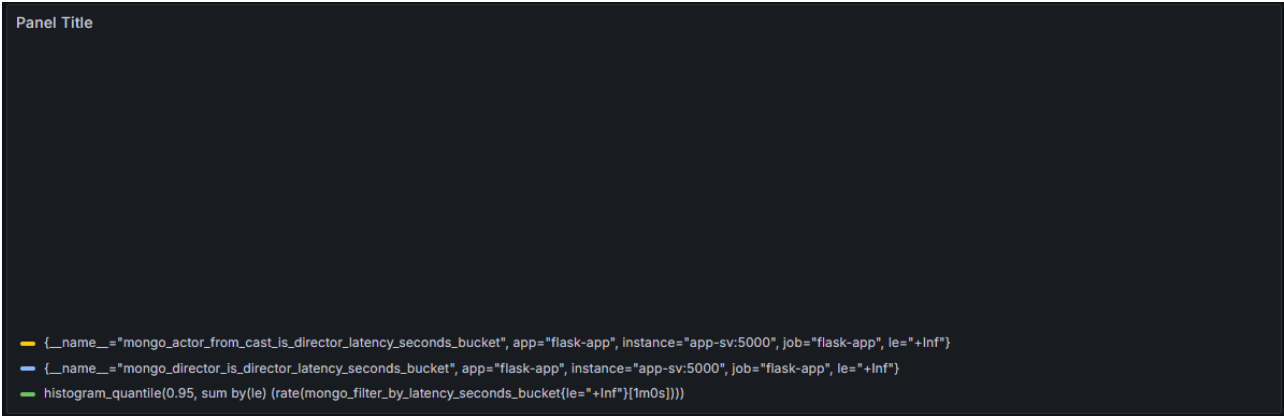
- Stat:



- Table:

Panel Title	
Time	histogram_quantile(0.95, sum by(le) (rate(mongo_filter_by_latency_seconds_bucket
2024-05-24 19:43:00	NaN
2024-05-24 19:43:15	NaN
2024-05-24 19:43:30	NaN
2024-05-24 19:43:45	NaN
2024-05-24 19:44:00	NaN
2024-05-24 19:44:15	NaN
2024-05-24 19:44:30	NaN
histogram_quantile(0.95, sum by(le) (rate(mongo_filter_by_latency_seconds_bucket{le="+Inf"}[1m0s])))	

- Pie chart:



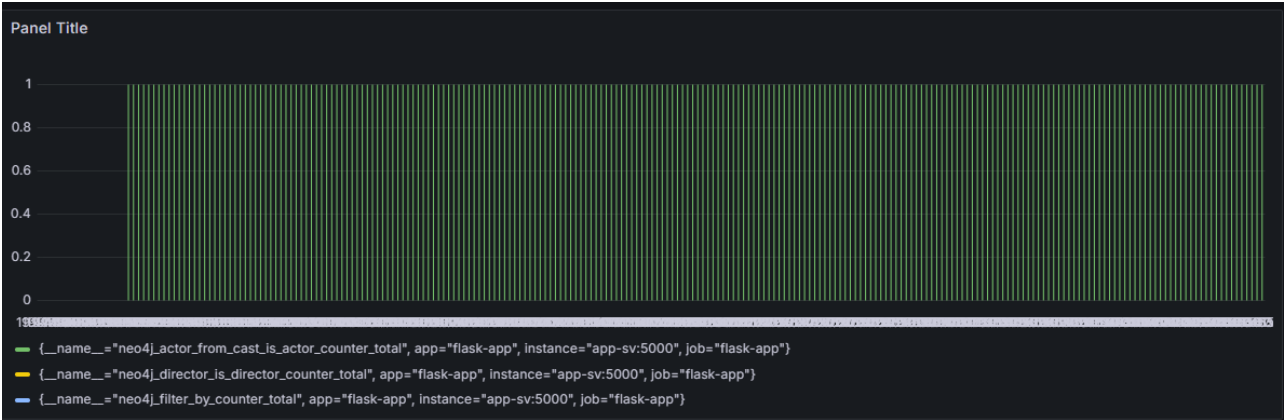
Las gráficas anteriores pareciera que no reflejan nada, incluso vemos unas vacías como Table y Pie, pero realmente hay datos, esto se logra apreciar gracias a gráficas como Time series y Bar chart, lo que deja la teoría de que los tiempos de consulta son tan rápidos y no logra generar las gráficas correspondientes, pero que sí existen.

3. neo4j_queries_sum:

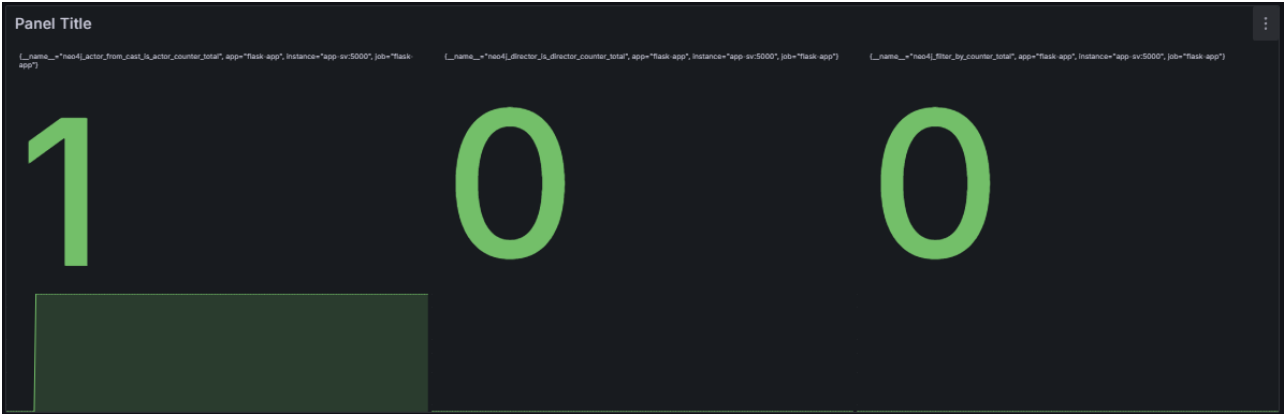
- Time series:



- Bar chart:



- Stat:

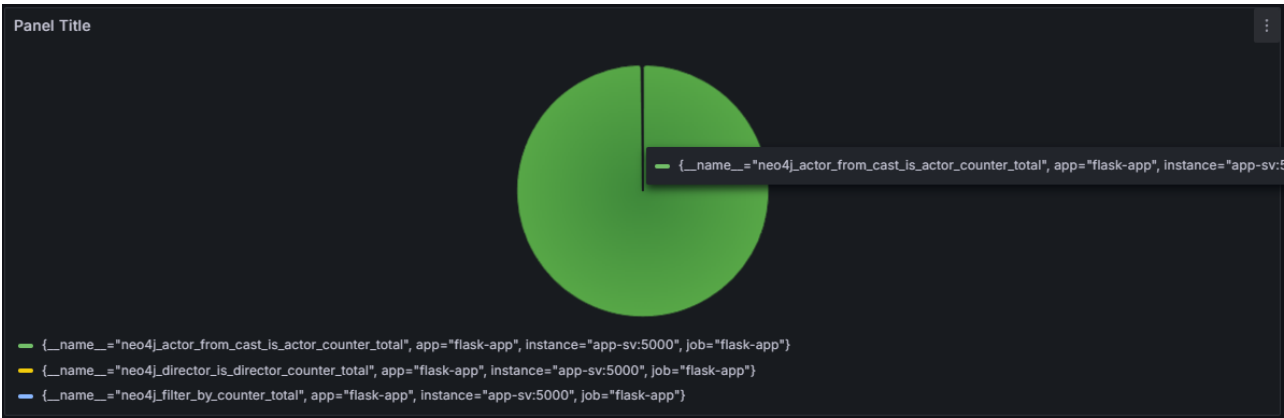


- Table:

Panel Title	
Time	{_name_='neo4j_actor_from_cast_is_actor_counter_total', app="flask-app", inst
2024-05-24 19:50:30	0
2024-05-24 19:50:45	0
2024-05-24 19:51:00	0
2024-05-24 19:51:15	1
2024-05-24 19:51:30	1
2024-05-24 19:51:45	1
2024-05-24 19:52:00	1

{_name_='neo4j_actor_from_cast_is_actor_counter_total', app="flask-app", instance="app-sv:5000", job="flask-app"} ▾

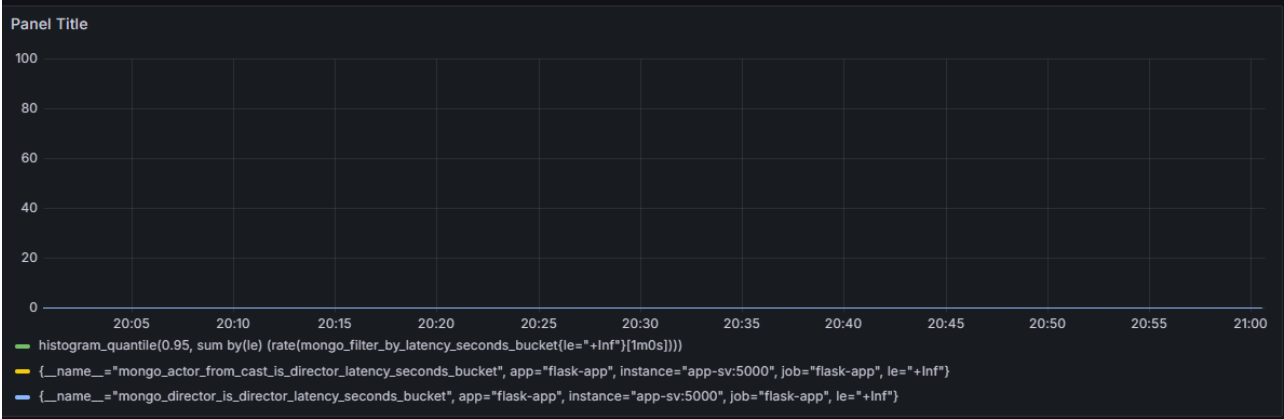
- Pie chart:



En este caso, las gráficas dejan en claro que se hizo una única consulta de búsqueda, principalmente en el área de filtrado. Aquí la que muestra la información con mayor claridad es la de Stat, ya que nos indica directamente el número de queries realizados en mongo. El caso particular respecto al de mongo, es que en la gráfica Table se sondeó un poco en la información hasta encontrar los datos, siendo reflejadas las consultas con su fecha y hora.

4. neo4j_queries_time:

- Time series:



- Bar chart:



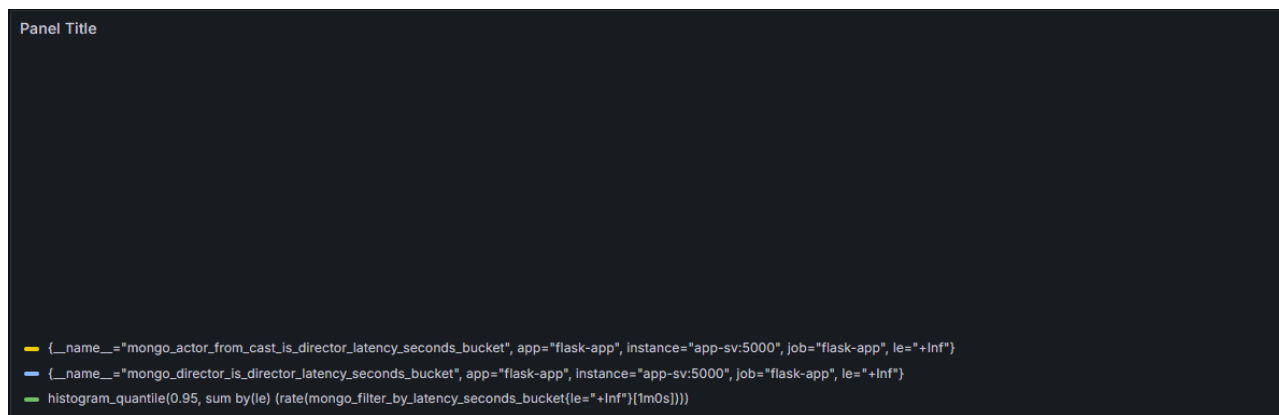
- Stat:



- Table:

Panel Title	
Time	histogram_quantile(0.95, sum by(le) (rate(mongo_filter_by_latency_seconds_bucket{le="+Inf"}[1m0s])))
2024-05-24 20:00:30	NaN
2024-05-24 20:00:45	NaN
2024-05-24 20:01:00	NaN
2024-05-24 20:01:15	NaN
2024-05-24 20:01:30	NaN
2024-05-24 20:01:45	NaN
2024-05-24 20:02:00	NaN
histogram_quantile(0.95, sum by(le) (rate(mongo_filter_by_latency_seconds_bucket{le="+Inf"}[1m0s])))	

- Pie chart:



Al igual que el de mongo, las gráficas anteriores pareciera que no reflejan nada, incluso vemos unas vacías como Table y Pie, pero realmente hay datos, esto se logra apreciar gracias a gráficas como Time series y Bar chart, lo que deja la teoría de que los tiempos de consulta son tan rápidos y no logra generar las gráficas correspondientes, pero que sí existen.

5. En conclusión, existen gráficas que son realmente útiles para reflejar el manejo de la información dentro del proceso automatizado, como lo es Stat, el cual indica los valores directamente en pantalla o incluso Table que muestra la cantidad de valores con su fecha y hora de ejecución, pero dichas gráficas parecen tener fallos a la hora de querer consultar valores más pequeños que unidades, mostrando supuestamente valores nulos, cuando en realidad aún existen valores, los cuales pueden ser vistos gracias a gráficas como Time series o Bar chart.
6. La interfaz de consultas se realiza mediante programación en bloques, lo que supone una experiencia interesante ya que siempre hemos programado con código como tal. Con esta programación uno aprende a realizar las cosas desde un punto de vista más gráfico y percatándose de lo que está realizando al momento.
7. Aunque ambas bases de datos contengan información de películas, no tienen la misma información, incluso hay algunas cuya información es incompleta. En el caso de Mongo, es muy completa, pero hay películas donde incluyen solo un grupo de los actores en vez de incluir a todos, o cómo Neo que incluye a todos los actores, pero su descripción de la película es muy incompleta.
8. Además de probar las bases de datos como Mongo y Neo, se prueba Firebase, la cual dice que es una base de datos en tiempo real, cosa que se puede evidenciar ya que a la hora de crear usuarios en el momento de ingresar por primera vez al sistema, este dato se muestra inmediatamente dentro de Firebase sin la necesidad de actualizar la página.
9. Para Mongo, es una base de datos interesante, tanto por ser NoSQL, como por el hecho de que puede ser manejada de manera gratuita desde el sitio web, en Atlas para ser específicos. Esto permite que sea una base de datos muy accesible y fácil de manejar comparada a otras como Elasticsearch que hay que montar tanto la base como a Kibana para poder manejarla.
10. La experiencia de utilizar nodos con Neo4j en vez de las tablas tradicionales es bastante única, porque las consultas se realizan según la entidad que deseas obtener y las relaciones que esta posee con otras entidades. Debido a esto las consultas terminan siendo bastante simples de realizar, el resultado de las consultas te puede devolver tanto las relaciones del nodo como los individuos con los que se relaciona, lo cual se puede observar de manera gráfica para facilitar su análisis.

Referencias

Docker. "Get Started | Docker". Docker. Accedido el 10 de abril de 2024. [En línea]. Disponible: <https://www.docker.com/get-started/>

Lens. "Lens | The Kubernetes IDE". Lens | The Kubernetes IDE. Accedido el 10 de abril de 2024. [En línea]. Disponible: <https://k8slens.dev/>

Python. "Download Python". Python.org. Accedido el 10 de abril de 2024. [En línea]. Disponible: <https://www.python.org/downloads/>

Helm. "Installing Helm". Helm. Accedido el 10 de abril de 2024. [En línea]. Disponible: <https://helm.sh/docs/intro/install/>

DevOps Made Easy. Install Helm On Windows. (24 de octubre de 2022). Accedido el 10 de abril de 2024. [Video en línea]. Disponible: <https://www.youtube.com/watch?v=ZKAIKoqlWac>

mattfarina. "Releases · helm/helm". GitHub. Accedido el 10 de abril de 2024. [En línea]. Disponible: <https://github.com/helm/helm/releases>

WinRAR. "Descargar WinRAR - Descarga gratuita recomendada". Soporte WinRAR - Sitio oficial WinRAR en español. Accedido el 1 de abril de 2024. [En línea]. Disponible: <https://www.winrar.es/descargas>